Lecture 11: Hidden Markov models, semi-supervised learning

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 27, 2026

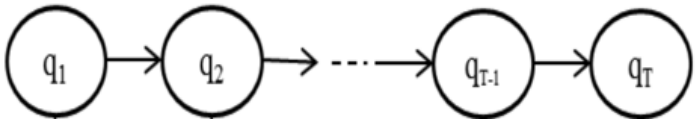
Announcements

- **Quiz 2:** Today at end of class
  - Tip: Read the questions carefully (nothing is meant to be tricky!)
- **Project topics:** Discuss with me in office hours today, TA office hours on Wednesday
- **Upcoming deadlines:** Project proposal 4/30, HW3 5/7

Hidden Markov models

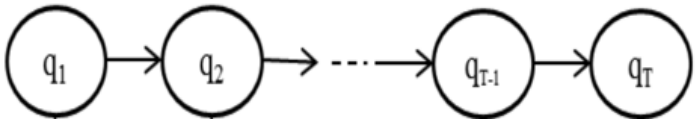
From n-gram LMs to hidden Markov models

n-gram LM as a graphical model:

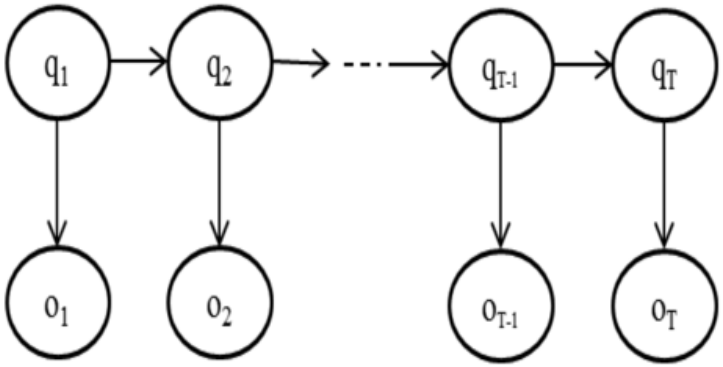


From n-gram LMs to hidden Markov models

n-gram LM as a graphical model:

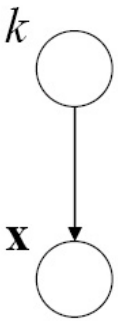


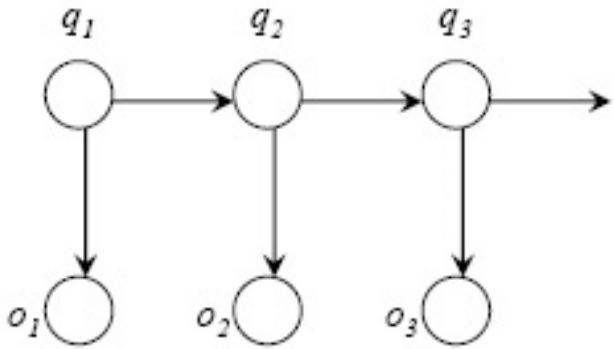
An HMM is like a language model where there is an n-gram over topics, and words are drawn independently given the topic:



From mixture models to HMMs

- HMMs are also an extension of mixture models, where the mixture components are not independent across data points
- Graphical model representation for mixture models and HMMs:





Elements of a (discrete) HMM

- N: Number of states; state at time t: $q_t \in \{1, \ldots, N\}$
- $V = \{v_1, \ldots, v_M\}$: Set of M possible observation labels (or vectors, in general); observation at time t: $o_t \in V$
- $\pi = \{\pi_i\}$: Initial state distribution, $\pi_i = P(q_1 = i), \;\; 1 \leq i \leq N$
- $\mathbf{A} = \{a_{ij}\}$: $N \times N$ state transition probability matrix, $a_{ij} = P(q_{t+1} = j | q_t = i), \;\; 1 \leq i, j, \leq N$
- $\mathbf{B} = \{b_i(k)\}$: Observation (or *emission*) distribution in state i, $b_i(k) = P(o_t = v_k | q_t = i), \;\; 1 \leq i \leq N, 1 \leq k \leq M$

The entire model can be denoted $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$

Elements of the "weather-mood HMM" example

- States: $N = 3$; let state 1 be F, state 2 be C, state 3 be R
- Obervation labels: $M = 2$; let $v_1 = H, v_2 = U$
- Model parameters $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$:

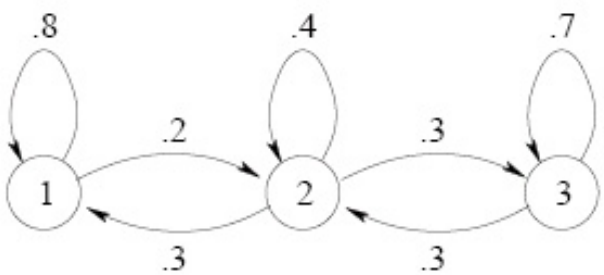
$$\pi = [0.4\ 0.3\ 0.3], \quad A = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.3 & 0.4 & 0.3 \\ 0 & 0.3 & 0.7 \end{bmatrix}, \quad B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

Elements of the "weather-mood HMM" example

- States: $N = 3$; let state 1 be F, state 2 be C, state 3 be R
- Obervation labels: $M = 2$; let $v_1 = H, v_2 = U$
- Model parameters $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$:

$$\pi = [0.4\ 0.3\ 0.3], \quad A = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.3 & 0.4 & 0.3 \\ 0 & 0.3 & 0.7 \end{bmatrix}, \quad B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

- $\mathbf{A}$ can also be represented via a *state transition diagram*:



The three basic HMM problems

The three basic HMM problems

1. *Scoring* (inference type 1): Given an observation sequence $\mathbf{O} = \{\mathbf{o}_1, \ldots, \mathbf{o}_T\}$ and a model $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$, how do we compute $P(\mathbf{O}|\lambda)$, the probability of the observation sequence?
   $\longrightarrow$ The forward & backward algorithms

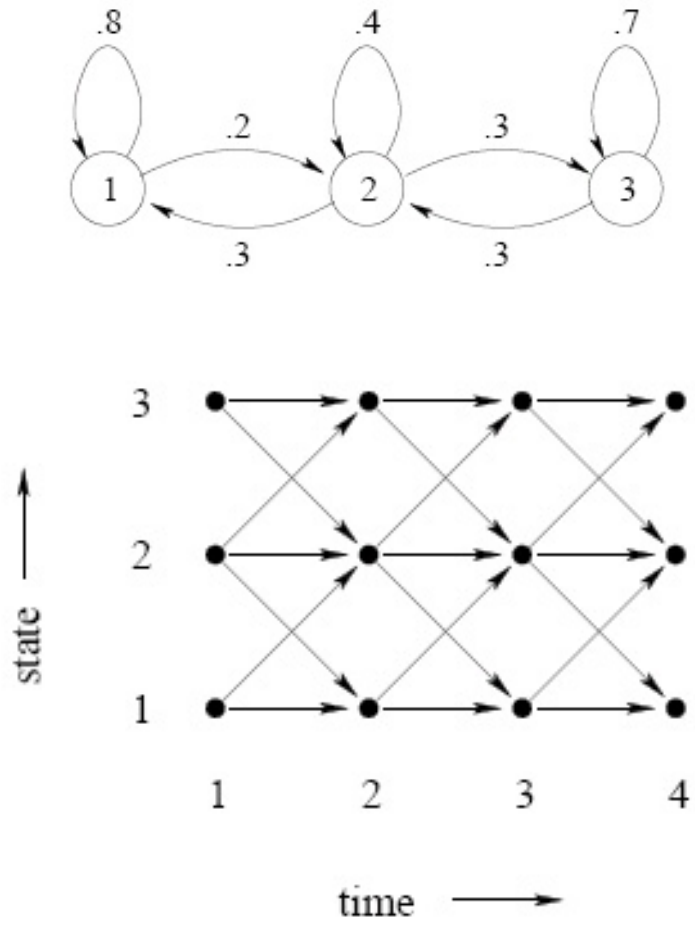
The three basic HMM problems

1. *Scoring* (inference type 1): Given an observation sequence $\mathbf{O} = \{\mathbf{o}_1, \ldots, \mathbf{o}_T\}$ and a model $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$, how do we compute $P(\mathbf{O}|\lambda)$, the probability of the observation sequence?
   $\longrightarrow$ The forward & backward algorithms
2. *Decoding* (inference type 2): Given an observation sequence $\mathbf{O} = \{o_1, \ldots, o_T\}$, how do we choose the state sequence $\mathbf{q} = \{q_1, \ldots, q_T\}$ most likely to have generated the observations?
   $\longrightarrow$ The Viterbi algorithm

The three basic HMM problems

1. *Scoring* (inference type 1): Given an observation sequence $\mathbf{O} = \{\mathbf{o}_1, \ldots, \mathbf{o}_T\}$ and a model $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$, how do we compute $P(\mathbf{O}|\lambda)$, the probability of the observation sequence?
   $\longrightarrow$ The forward & backward algorithms
2. *Decoding* (inference type 2): Given an observation sequence $\mathbf{O} = \{o_1, \ldots, o_T\}$, how do we choose the state sequence $\mathbf{q} = \{q_1, \ldots, q_T\}$ most likely to have generated the observations?
   $\longrightarrow$ The Viterbi algorithm
3. *Training* (learning): Given a training set of observations $\mathbf{O}$, how do we set the model parameters $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$ to maximize $P(\mathbf{O}|\lambda)$ (maximum-likelihood estimation)
   $\longrightarrow$ The Baum-Welch algorithm (EM applied to HMMs)

Representing transition diagram with a trellis



This representation is useful in visualizing the algorithms involved in solving the three problems

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$
- For $t > 1$: Sum over all ways of getting to current state at time t:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$

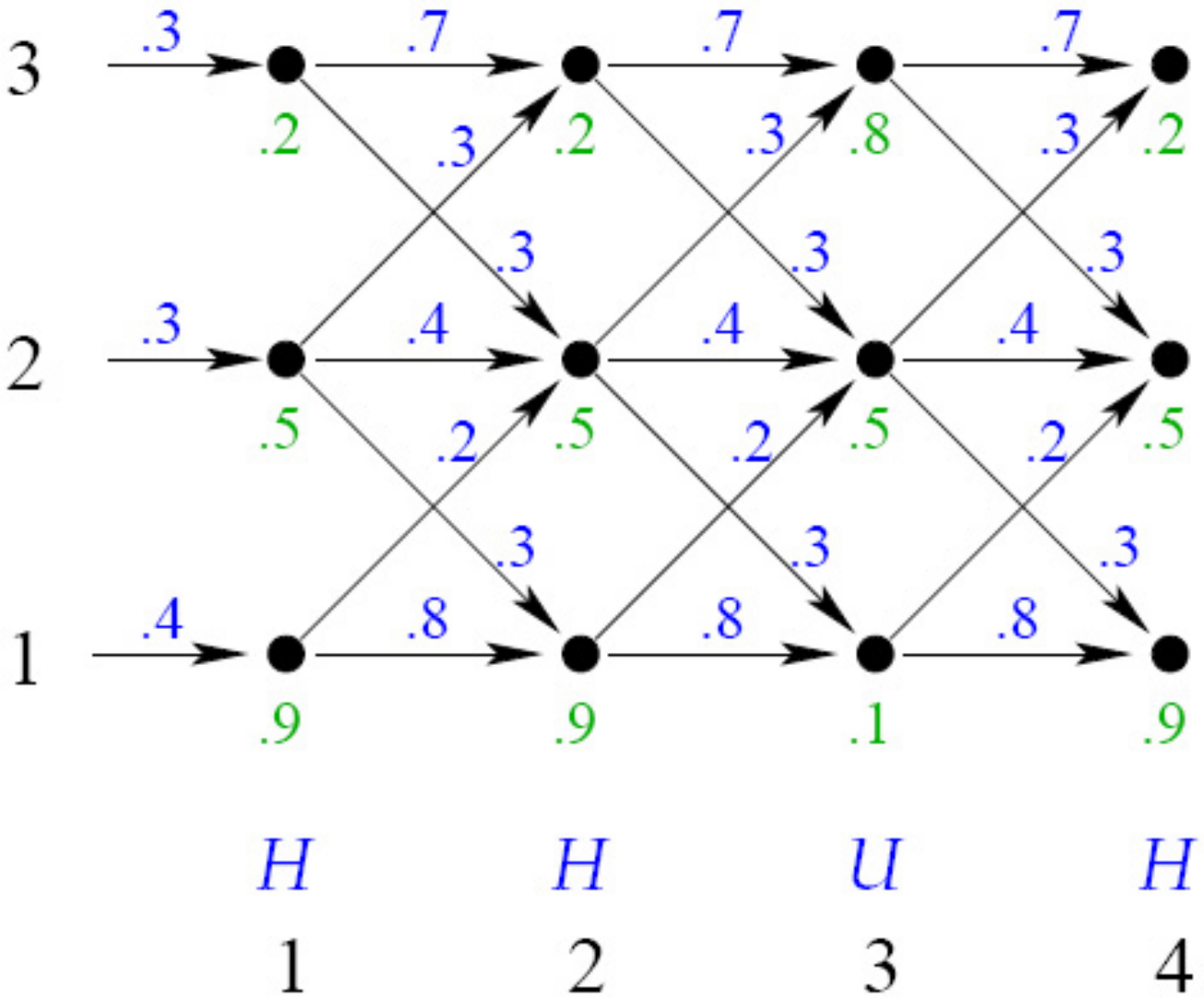
Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$
- For $t > 1$: Sum over all ways of getting to current state at time t:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$

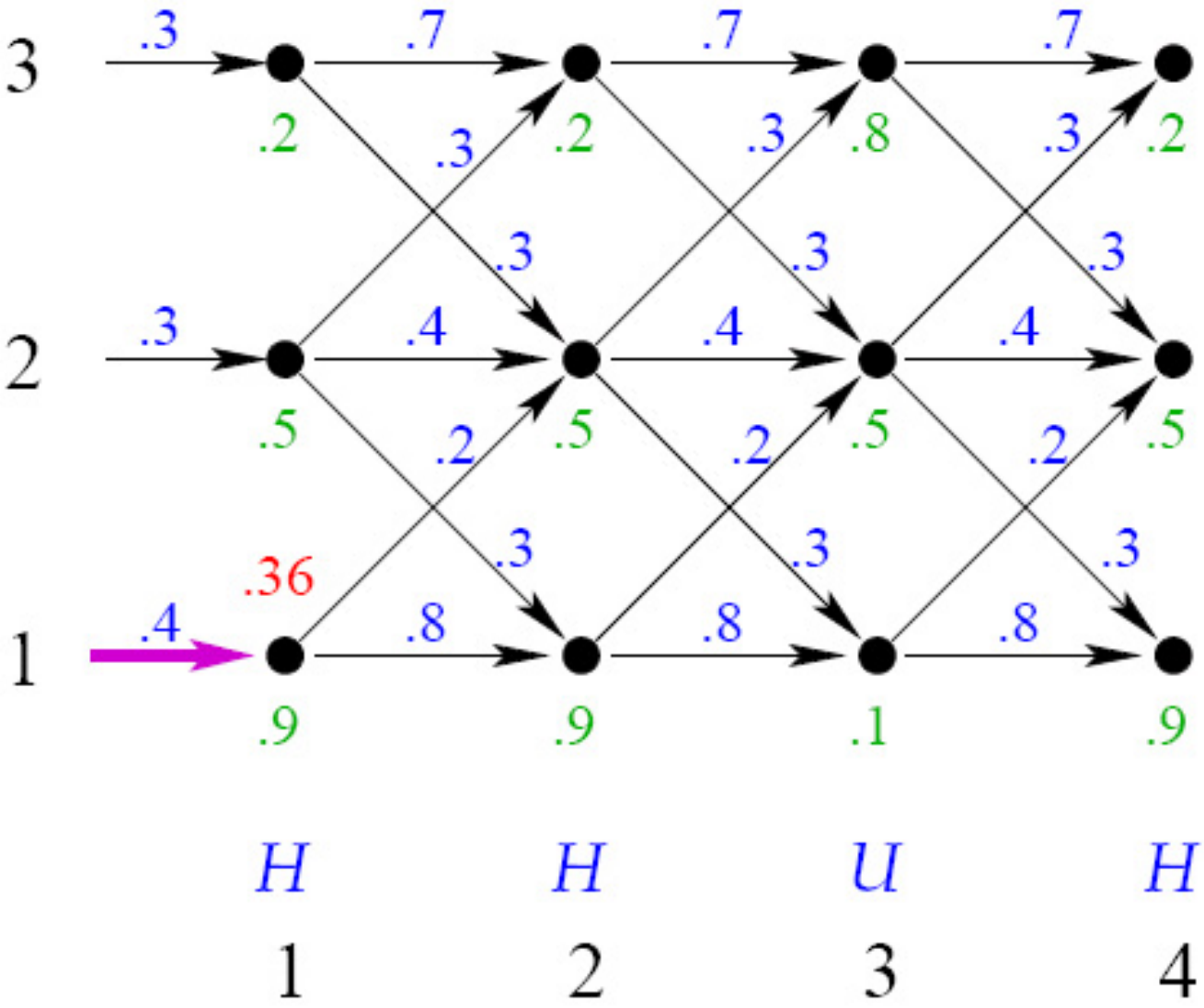
Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1\mathbf{o}_2 \ldots \mathbf{o}_t, q_t = i|\lambda), \;\; 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$
- For $t > 1$: Sum over all ways of getting to current state at time t:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij}\right] b_j(\mathbf{o}_t)$$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$
- Calculation is on the order of N^2T. $N = 5, T = 100 \Rightarrow \sim 2500$ computations, instead of 10^{72}

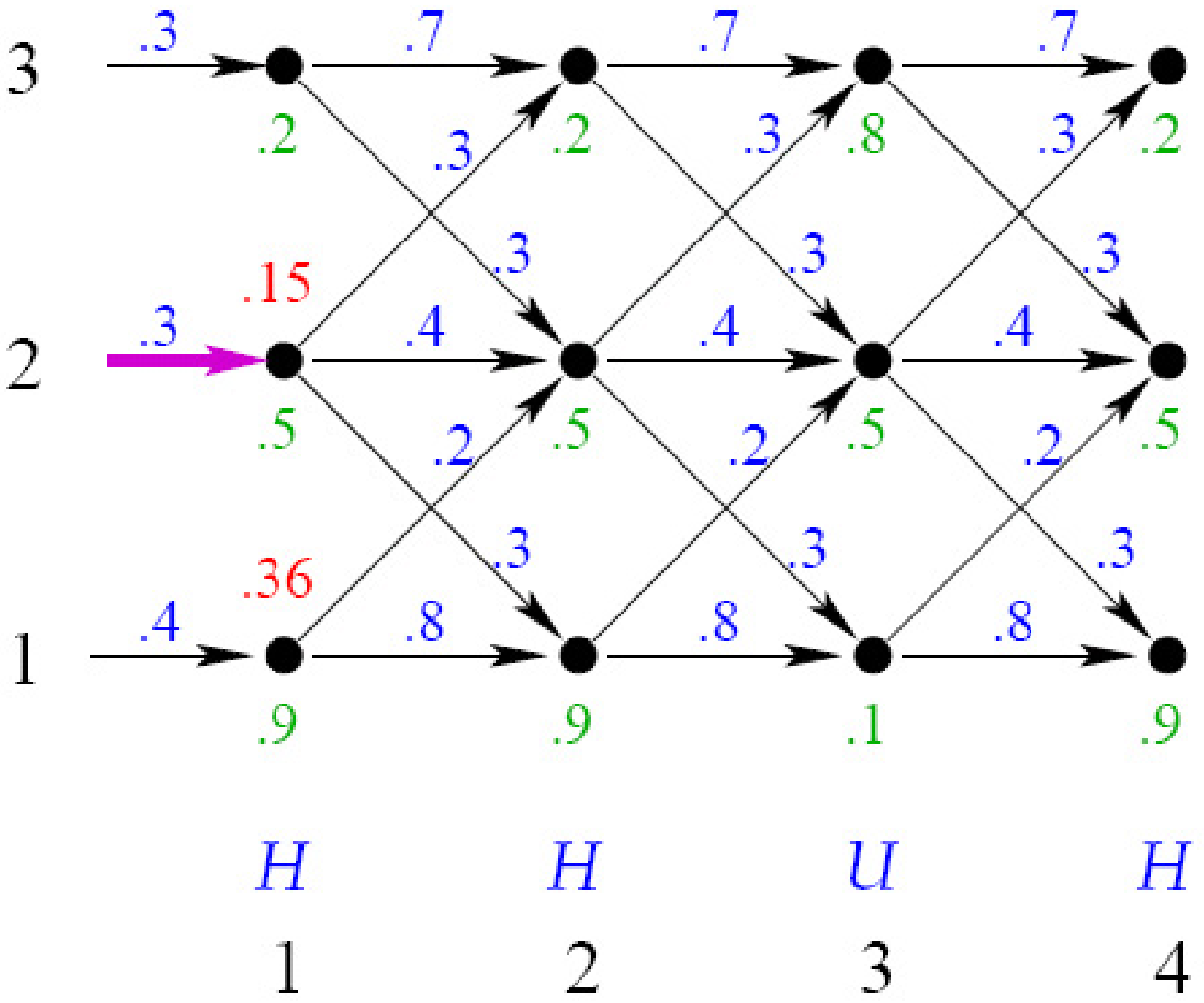
The forward algorithm: Example



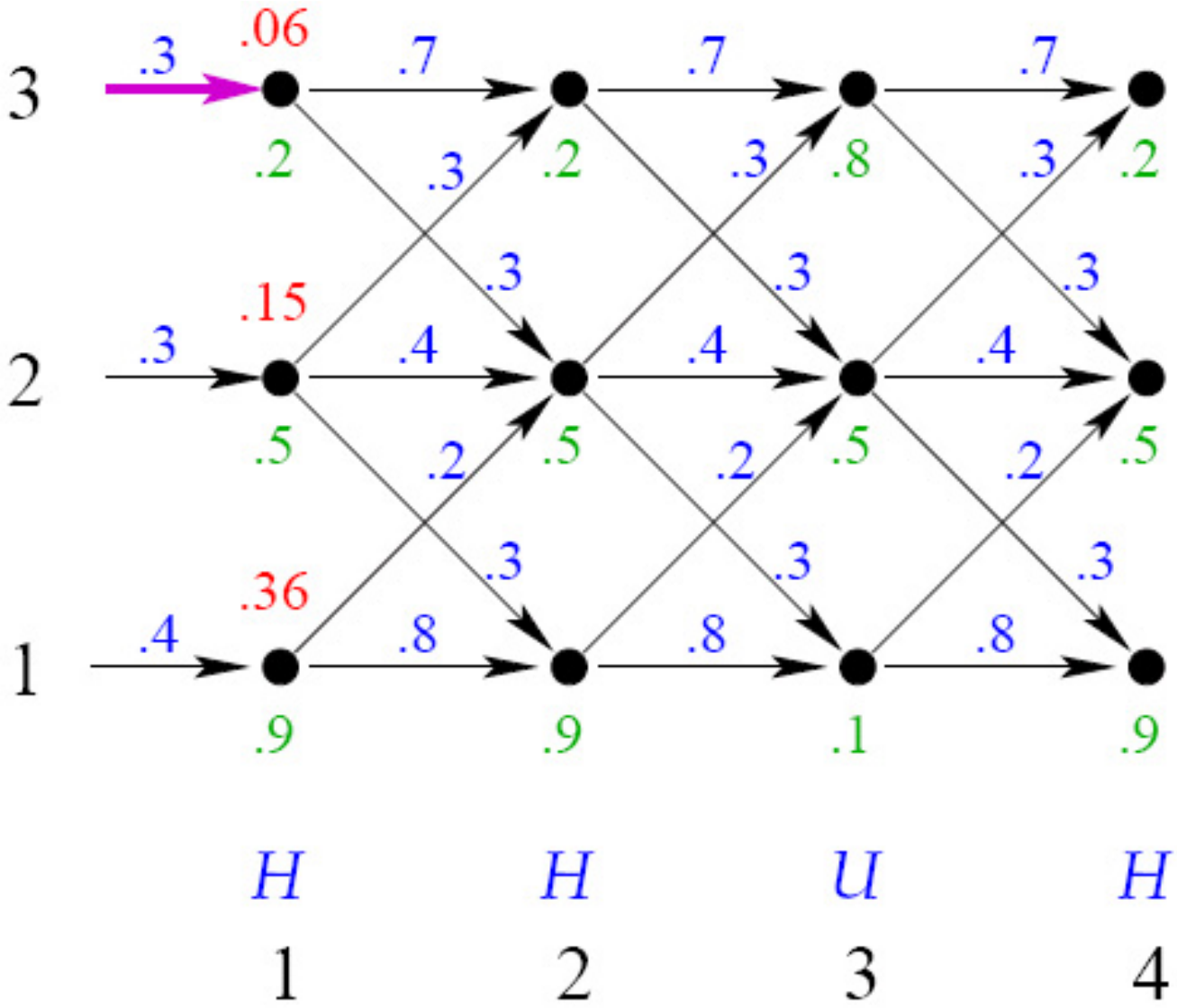
The forward algorithm: Example



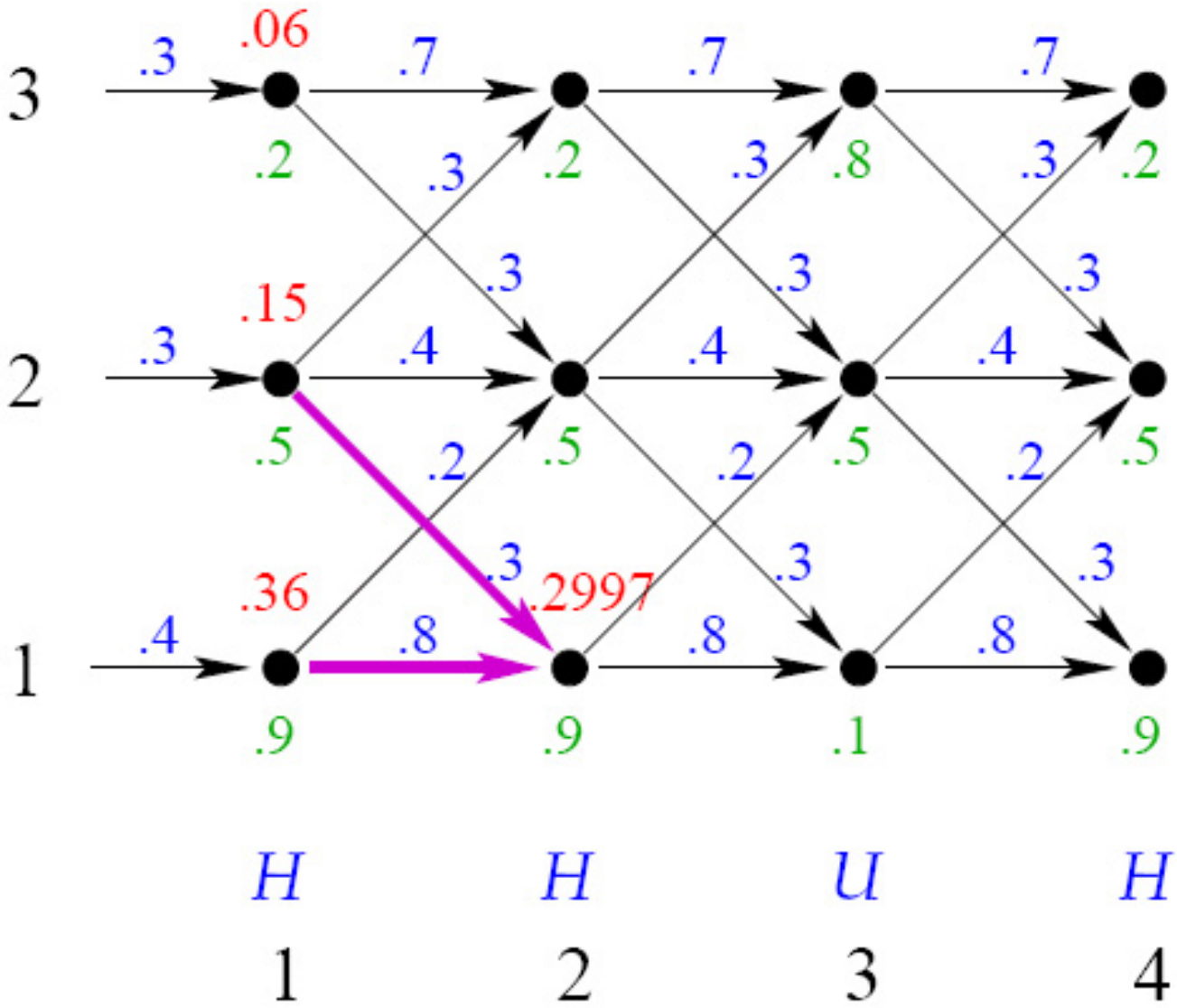
The forward algorithm: Example



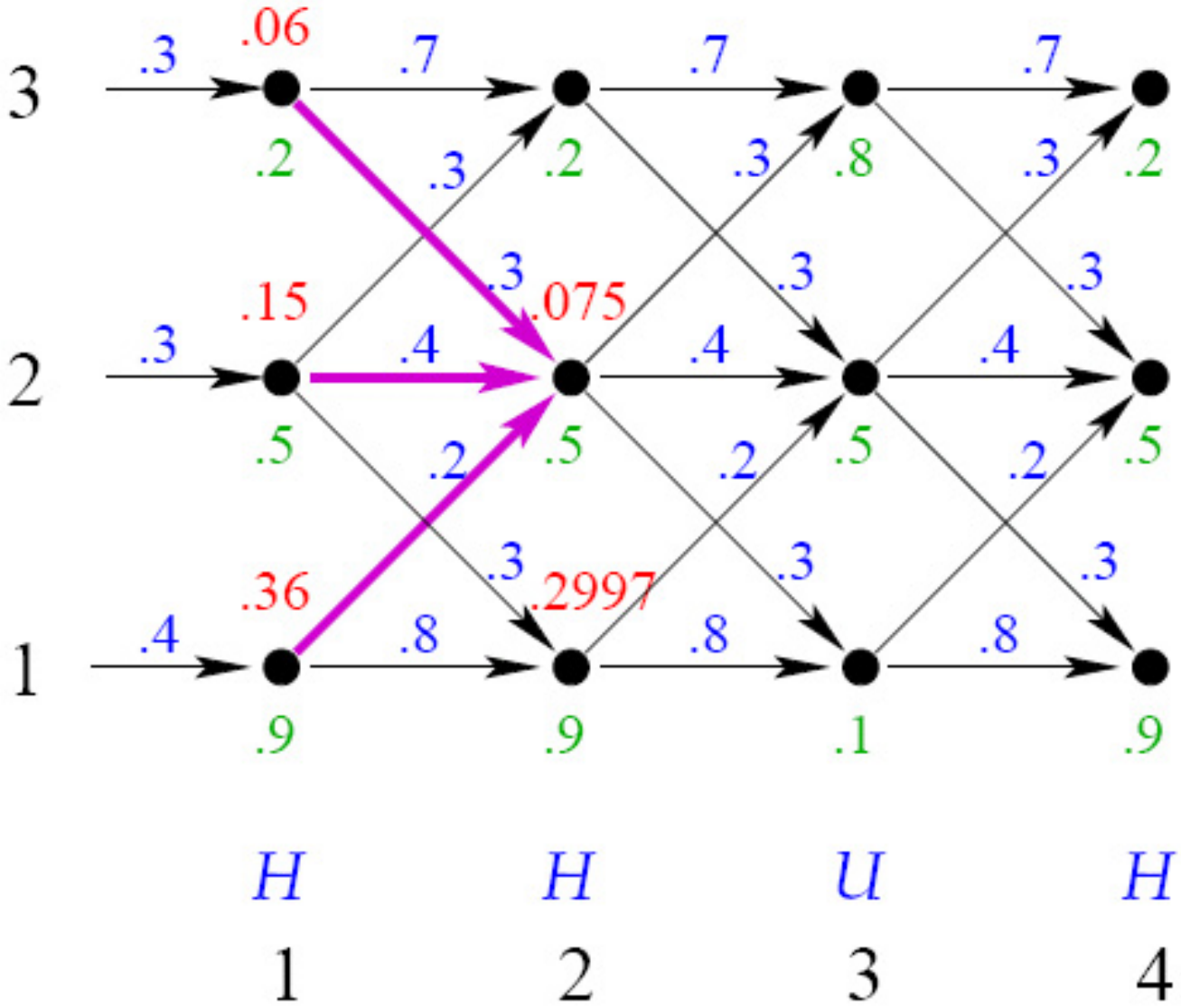
The forward algorithm: Example



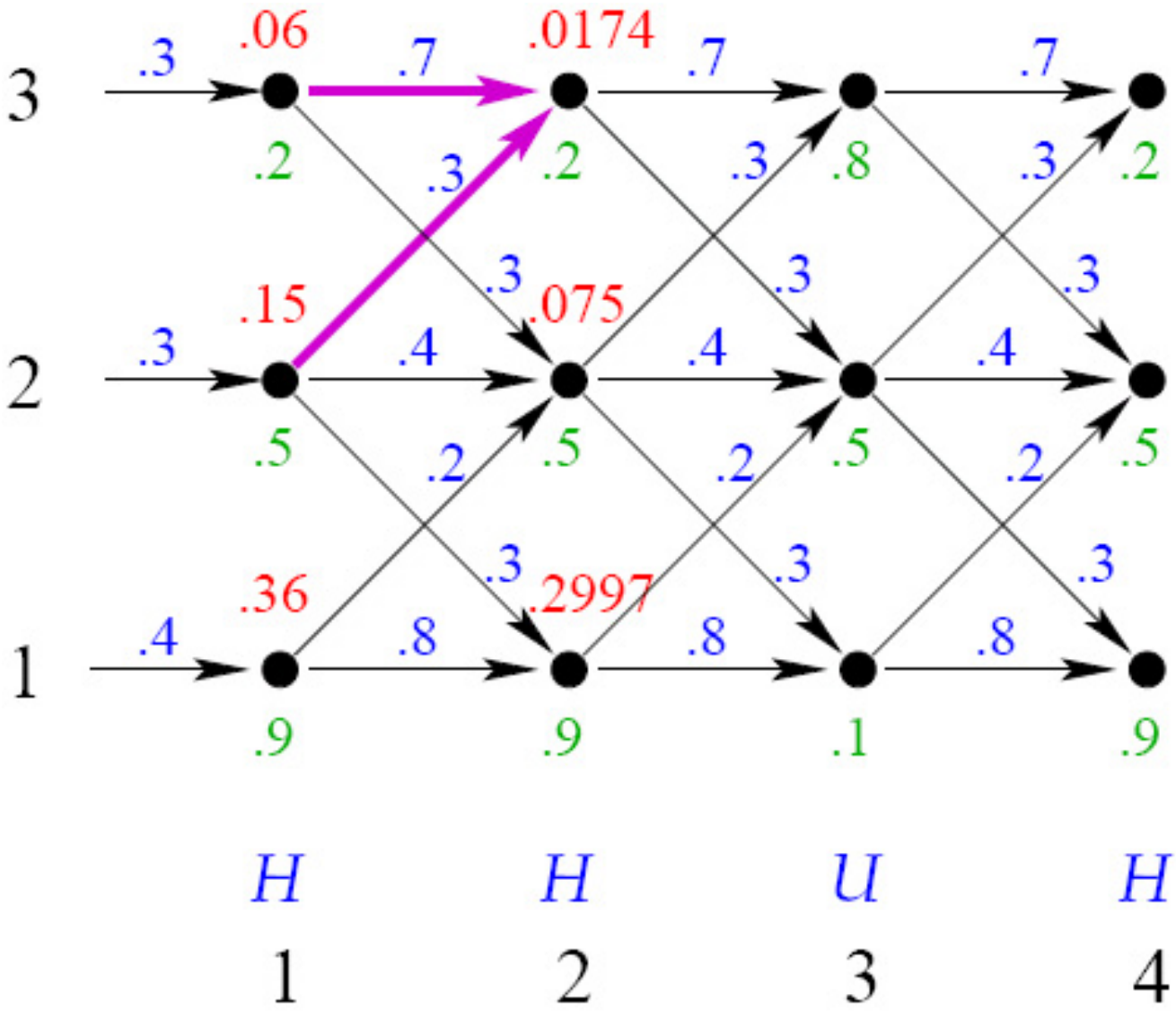
The forward algorithm: Example



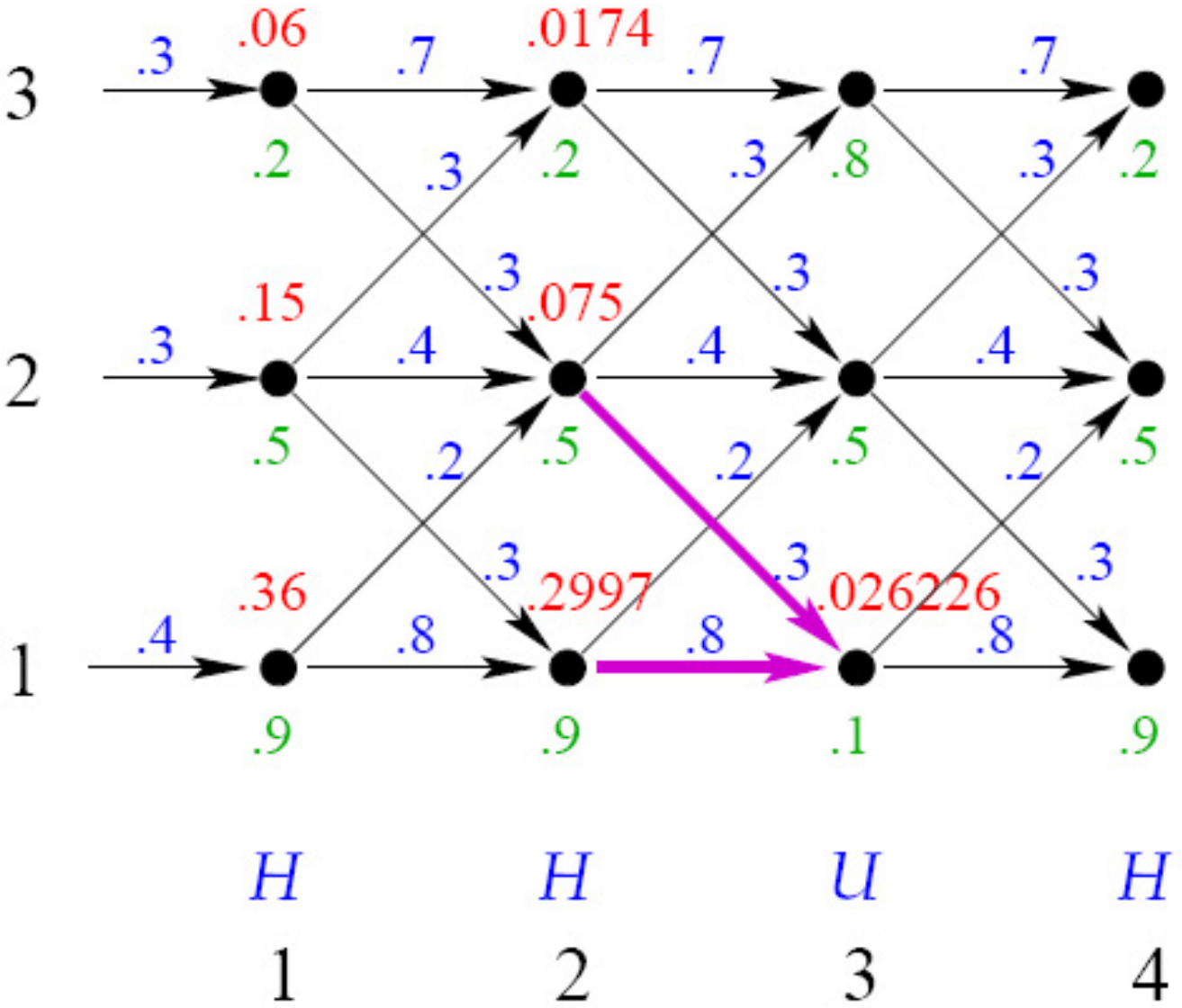
The forward algorithm: Example



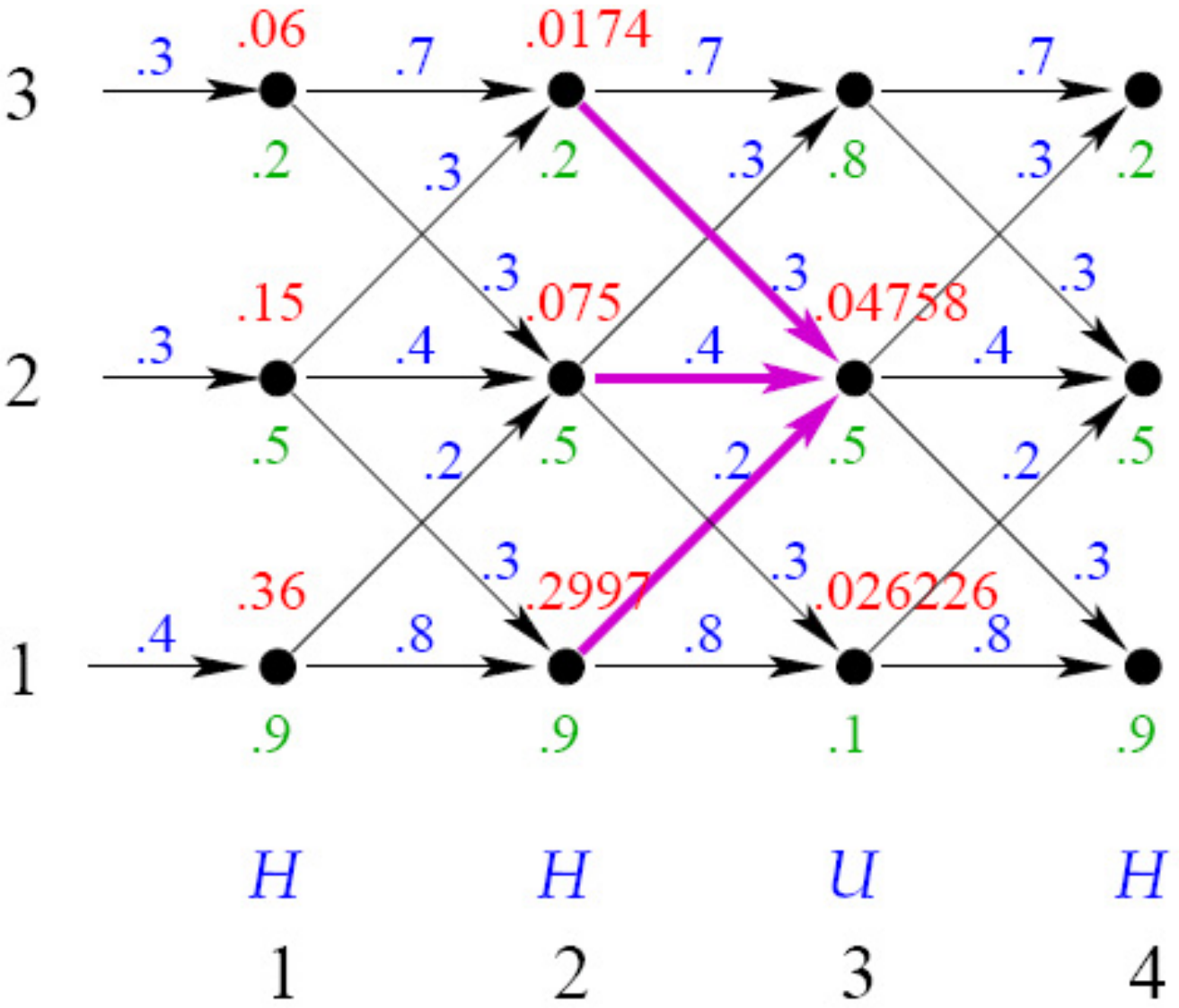
The forward algorithm: Example



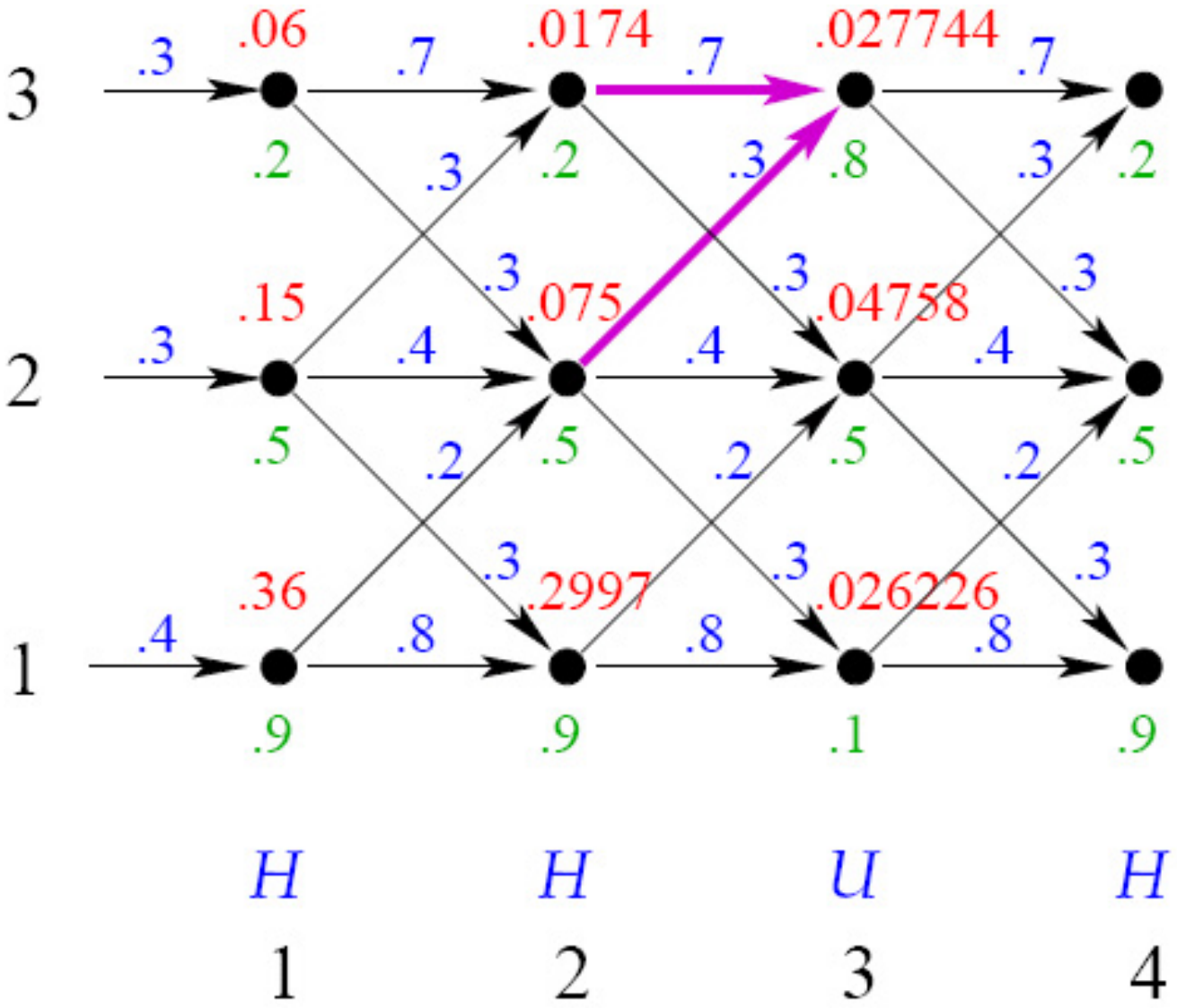
The forward algorithm: Example



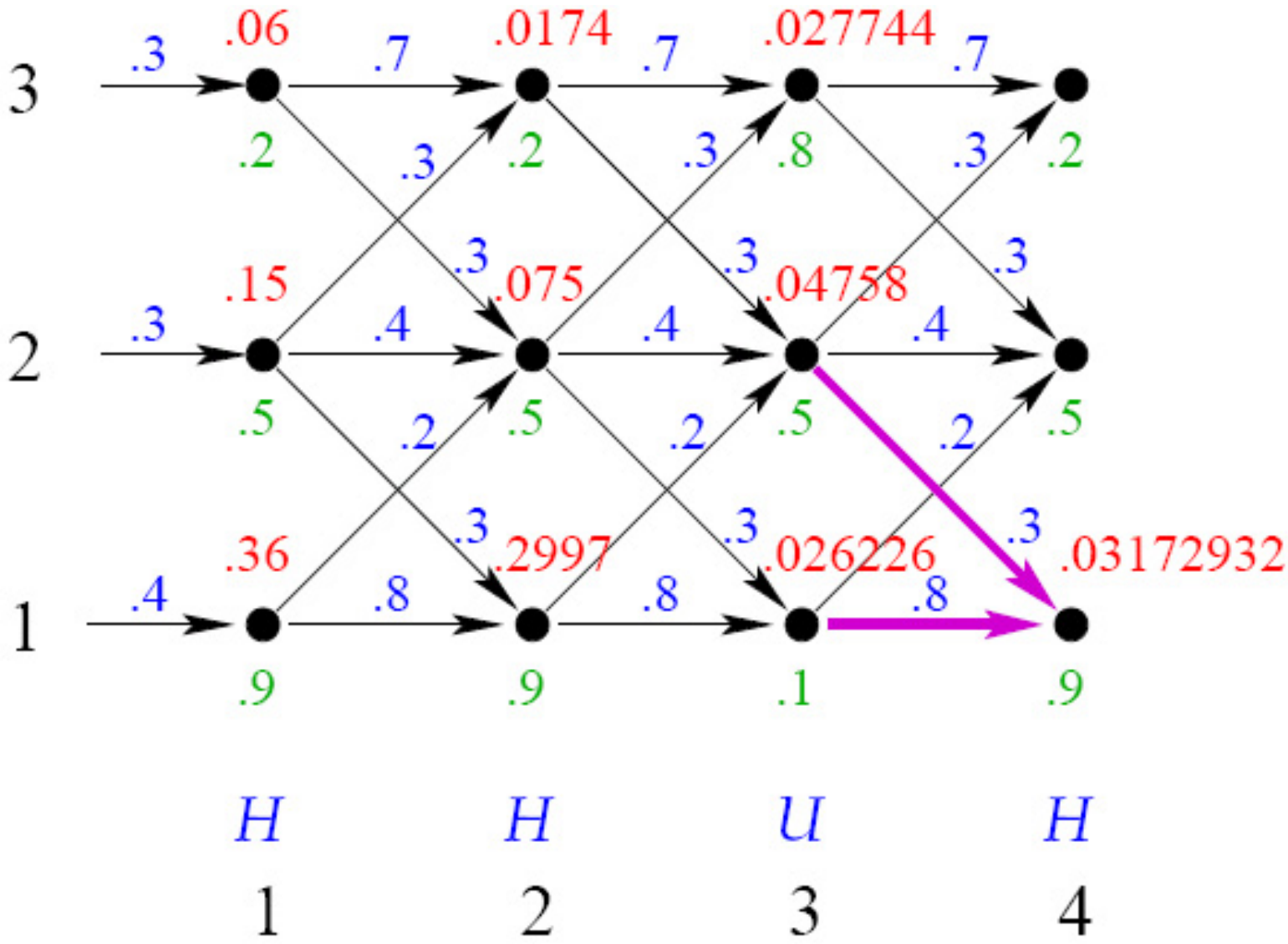
The forward algorithm: Example



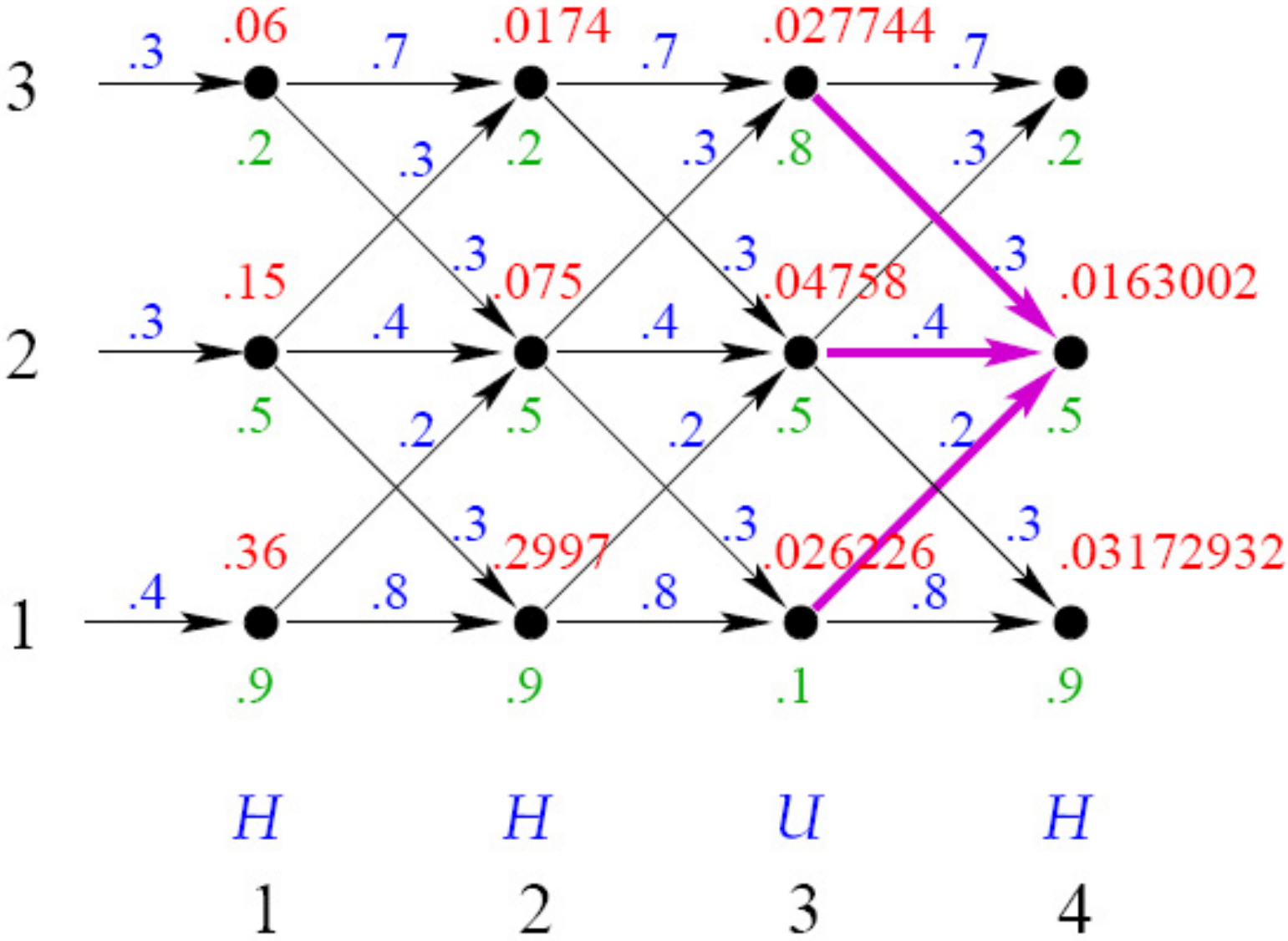
The forward algorithm: Example



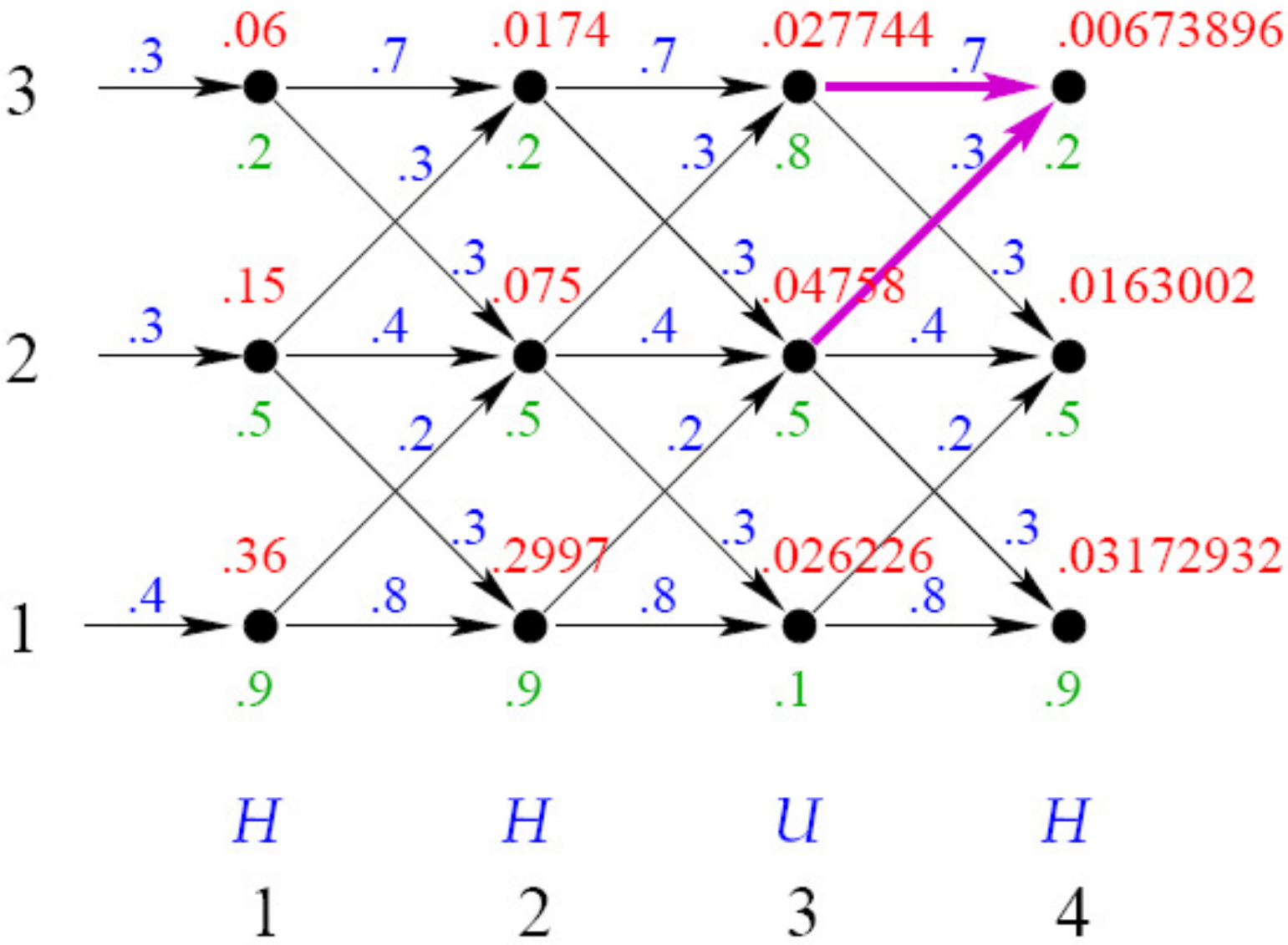
The forward algorithm: Example



The forward algorithm: Example



The forward algorithm: Example



Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \pi_i b_i(\mathbf{o}_1) \beta_1(i)$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \pi_i b_i(\mathbf{o}_1) \beta_1(i)$
- As in forward algorithm, computation is N^2T

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \pi_i b_i(\mathbf{o}_1) \beta_1(i)$
- As in forward algorithm, computation is N^2T
- Either algorithm alone can be used to compute $p(\mathbf{O}|\lambda)$, but both αs and βs will be needed for the training problem

Solving the decoding problem: The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\cdots,q_{t-1}} P(q_1 q_2 \ldots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t | \lambda)$

Solving the decoding problem: The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\cdots,q_{t-1}} P(q_1 q_2 \ldots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t | \lambda)$
- Initialization: $\delta_1(i) = \pi_i b_i(\mathbf{o}_1)$ (Note: this $= \alpha_1(i)$)

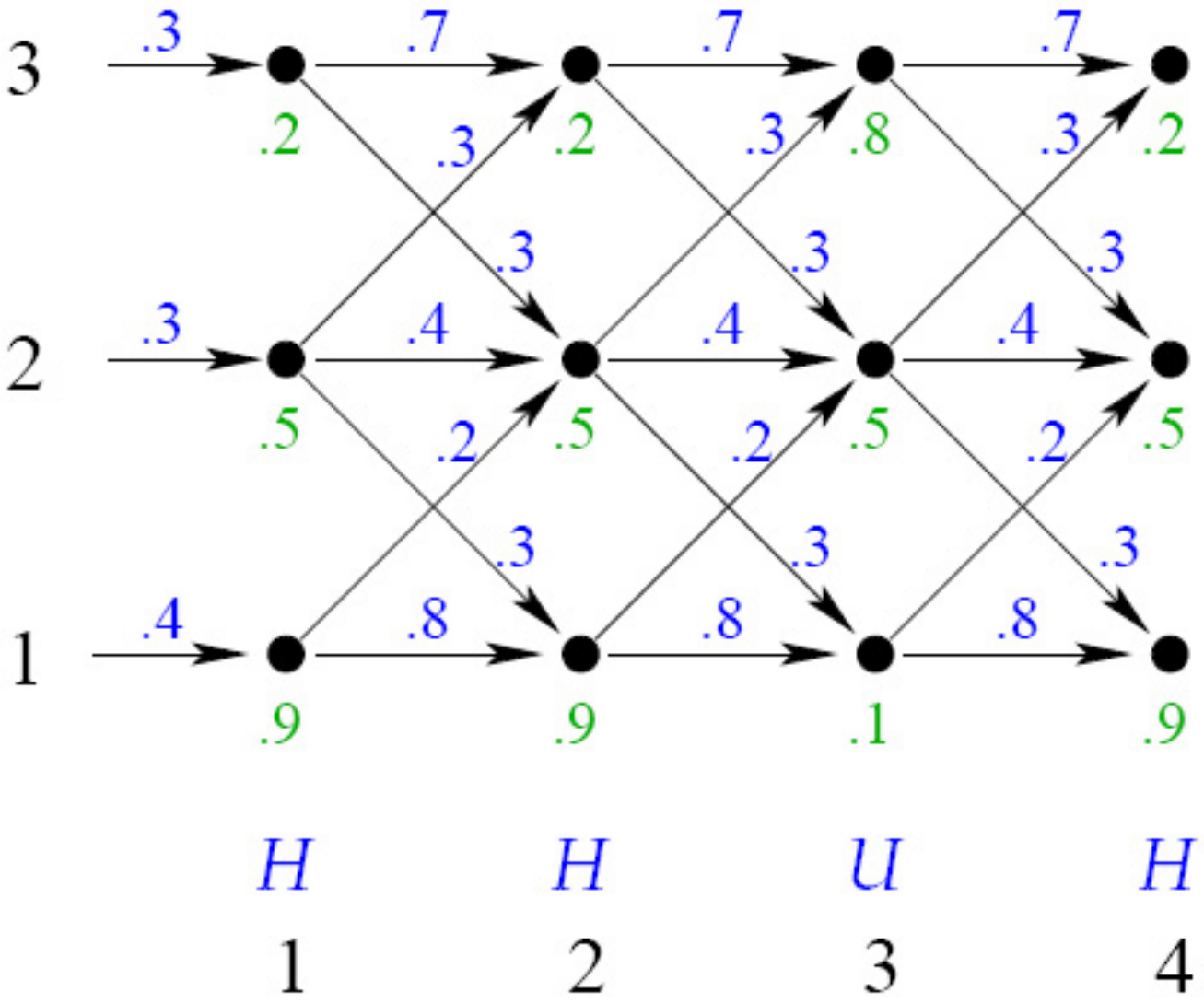
Solving the decoding problem: The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\cdots,q_{t-1}} P(q_1 q_2 \ldots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t | \lambda)$
- Initialization: $\delta_1(i) = \pi_i b_i(\mathbf{o}_1)$ (Note: this $= \alpha_1(i)$)
- Recursion: $\delta_t(j) = \max_{1 \le i \le N} \left[\delta_{t-1}(i) a_{ij}\right] b_j(\mathbf{o}_t), \quad 2 \le t \le T$
- Note the similarity to the α recursion:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij}\right] b_j(\mathbf{o}_t)$$
  - Same as the forward algorithm, except we keep the max rather than the sum at each node!

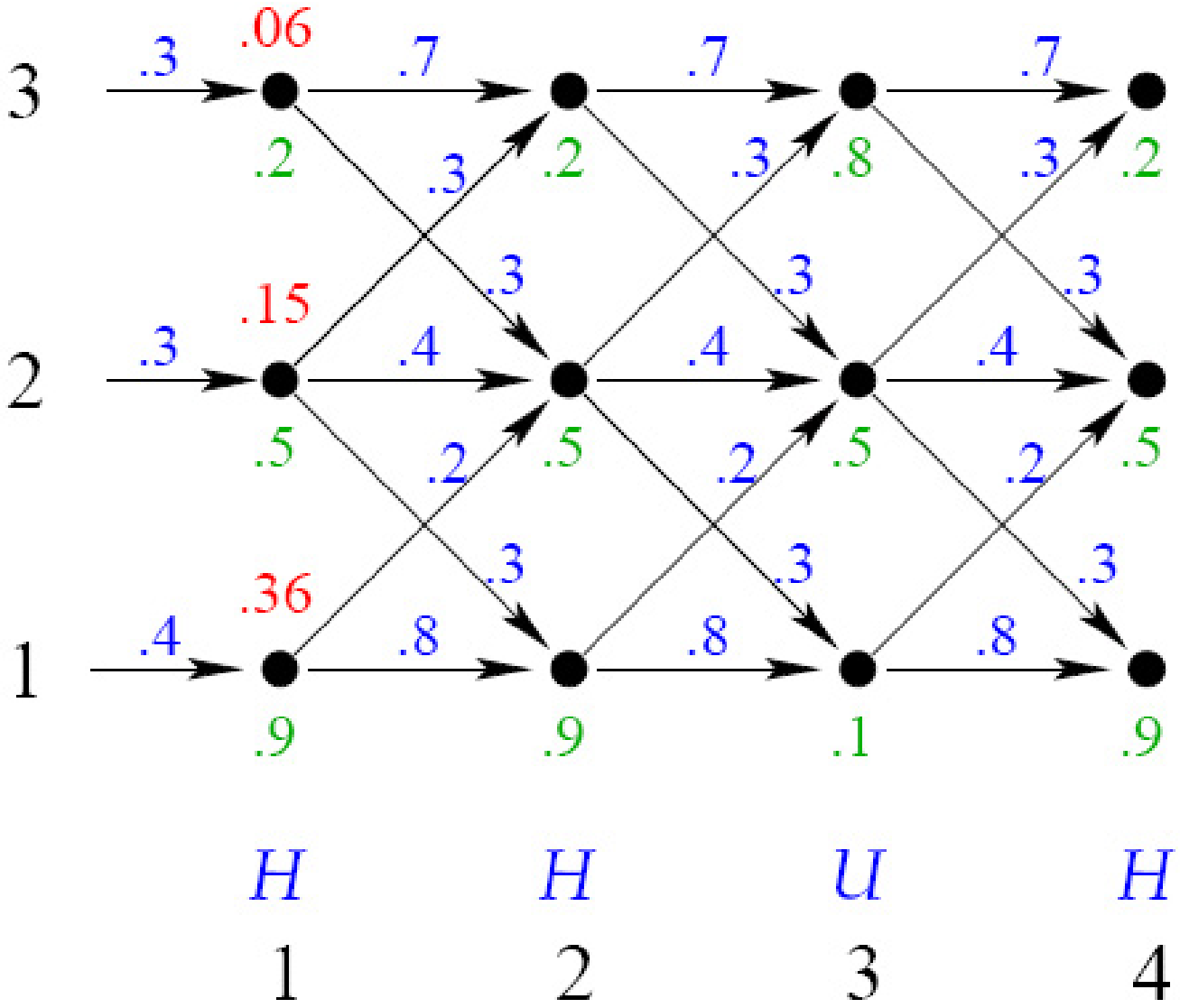
Solving the decoding problem: The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\cdots,q_{t-1}} P(q_1 q_2 \ldots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t | \lambda)$
- Initialization: $\delta_1(i) = \pi_i b_i(\mathbf{o}_1)$ (Note: this $= \alpha_1(i)$)
- Recursion: $\delta_t(j) = \max_{1 \le i \le N} [\delta_{t-1}(i) a_{ij}] \, b_j(\mathbf{o}_t), \quad 2 \le t \le T$
- Note the similarity to the α recursion:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$
  - Same as the forward algorithm, except we keep the max rather than the sum at each node!
  - To retrieve the best path, also need to keep a back-pointer from each node to the best incoming partial path
- As in forward algorithm, computation is $\approx N^2 T$

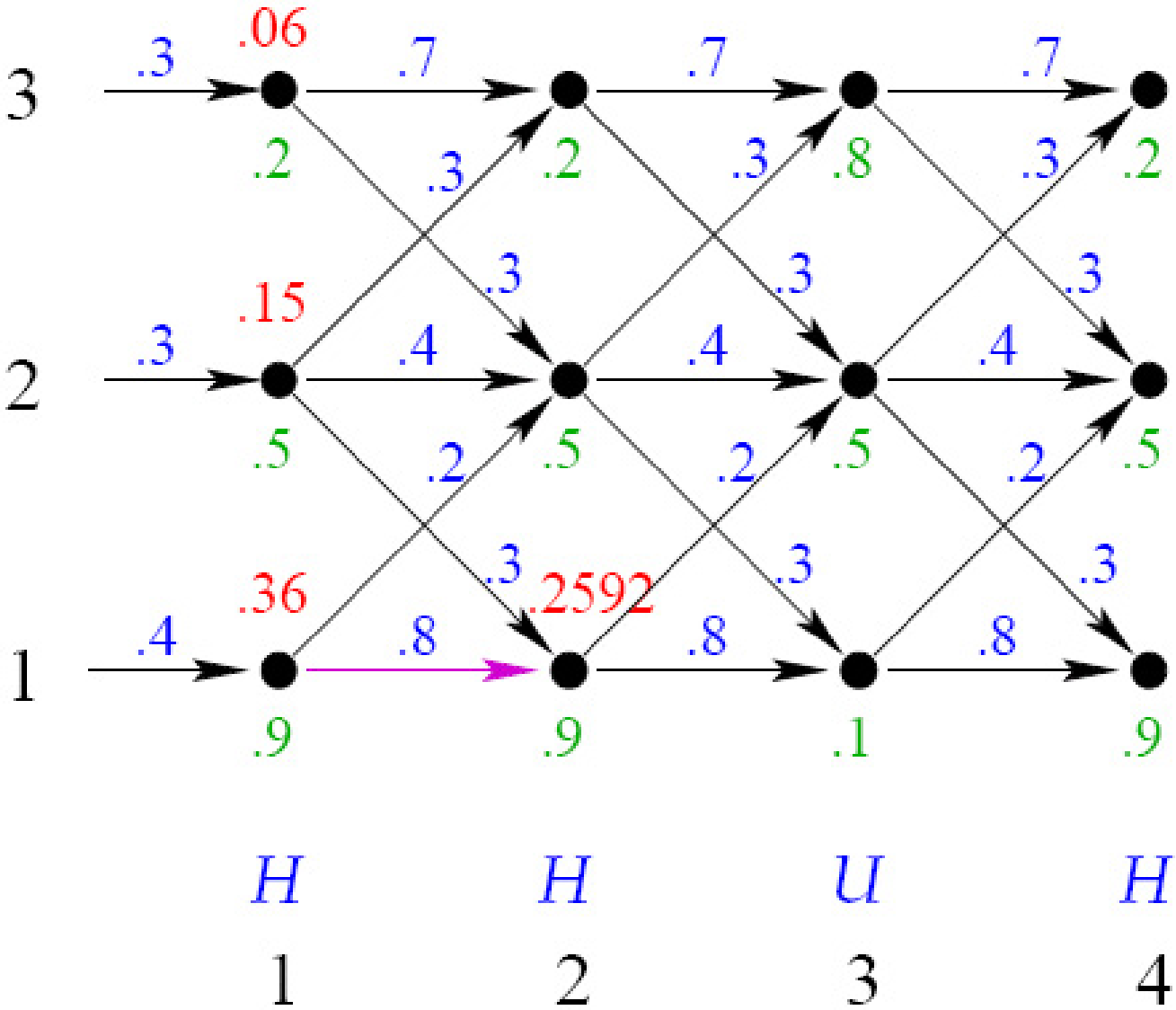
The Viterbi algorithm: Example



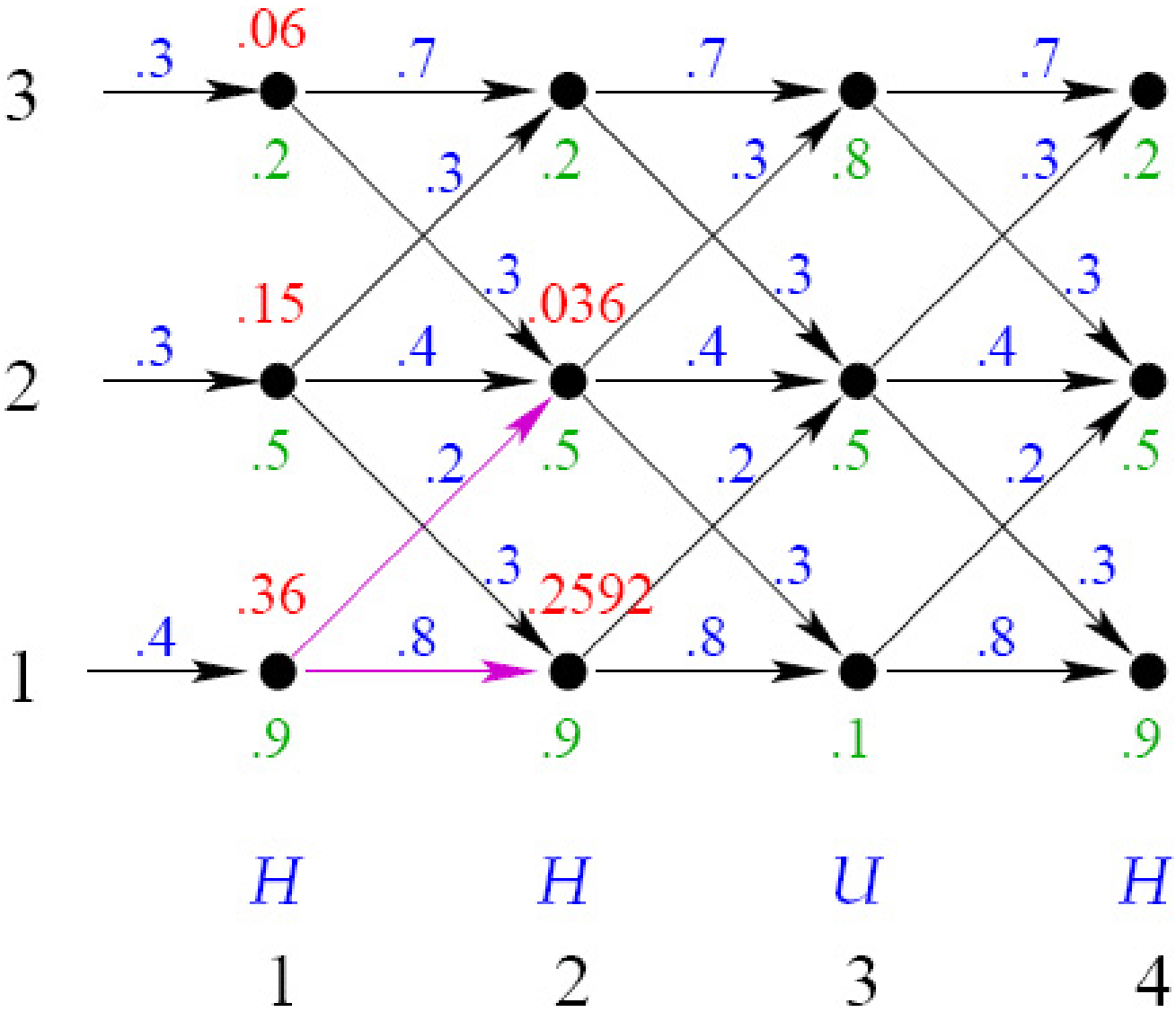
The Viterbi algorithm: Example



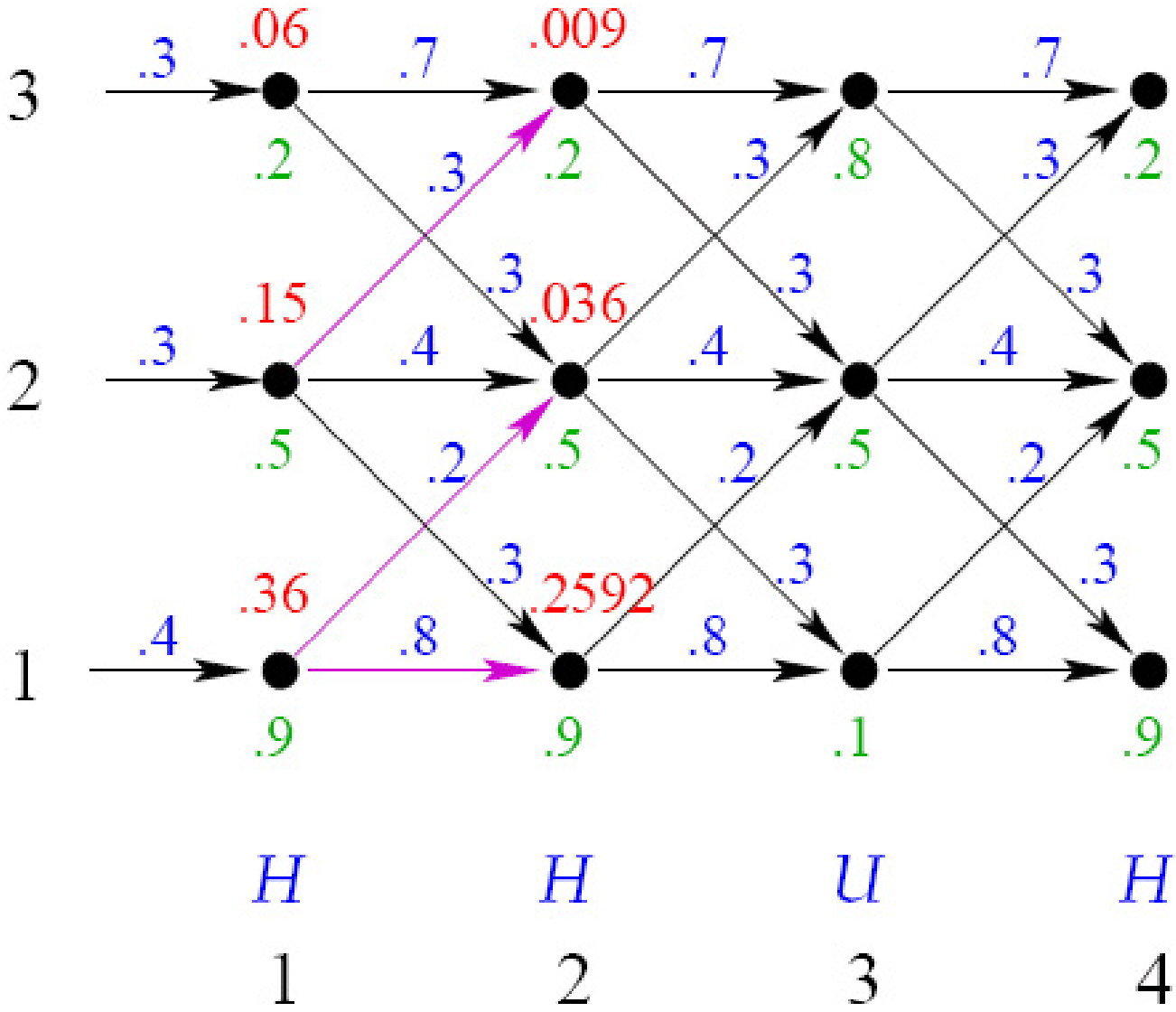
The Viterbi algorithm: Example



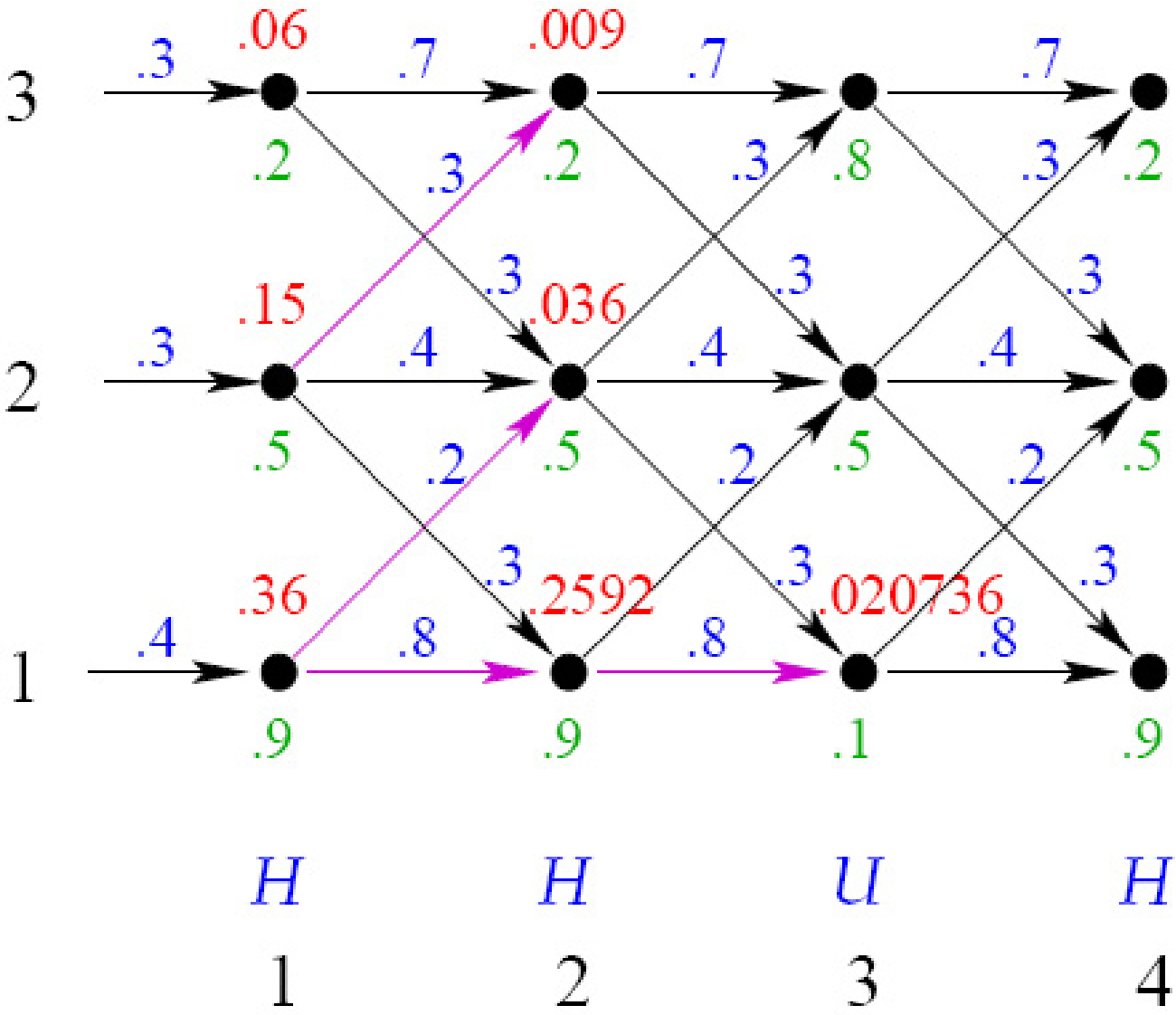
The Viterbi algorithm: Example



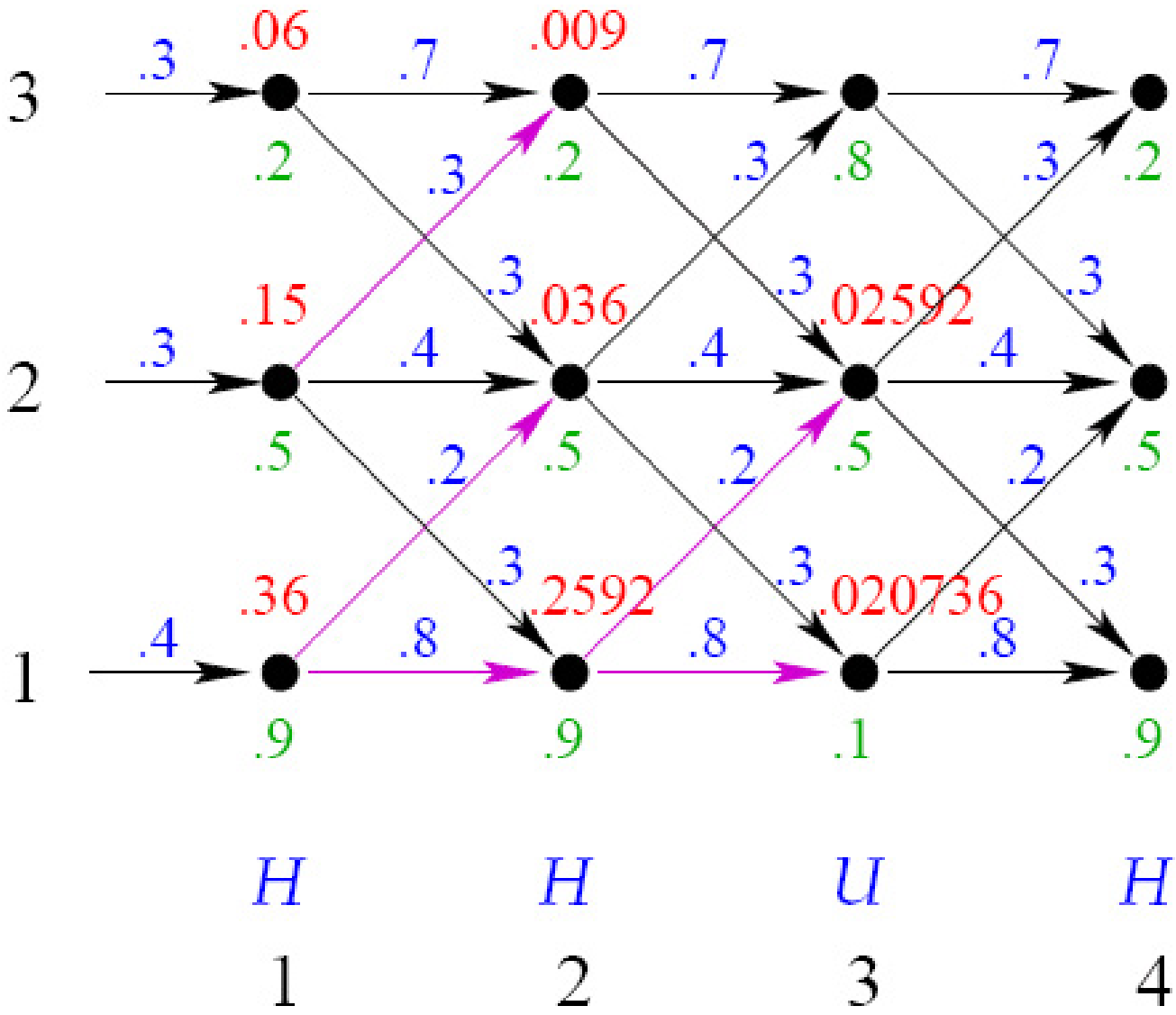
The Viterbi algorithm: Example



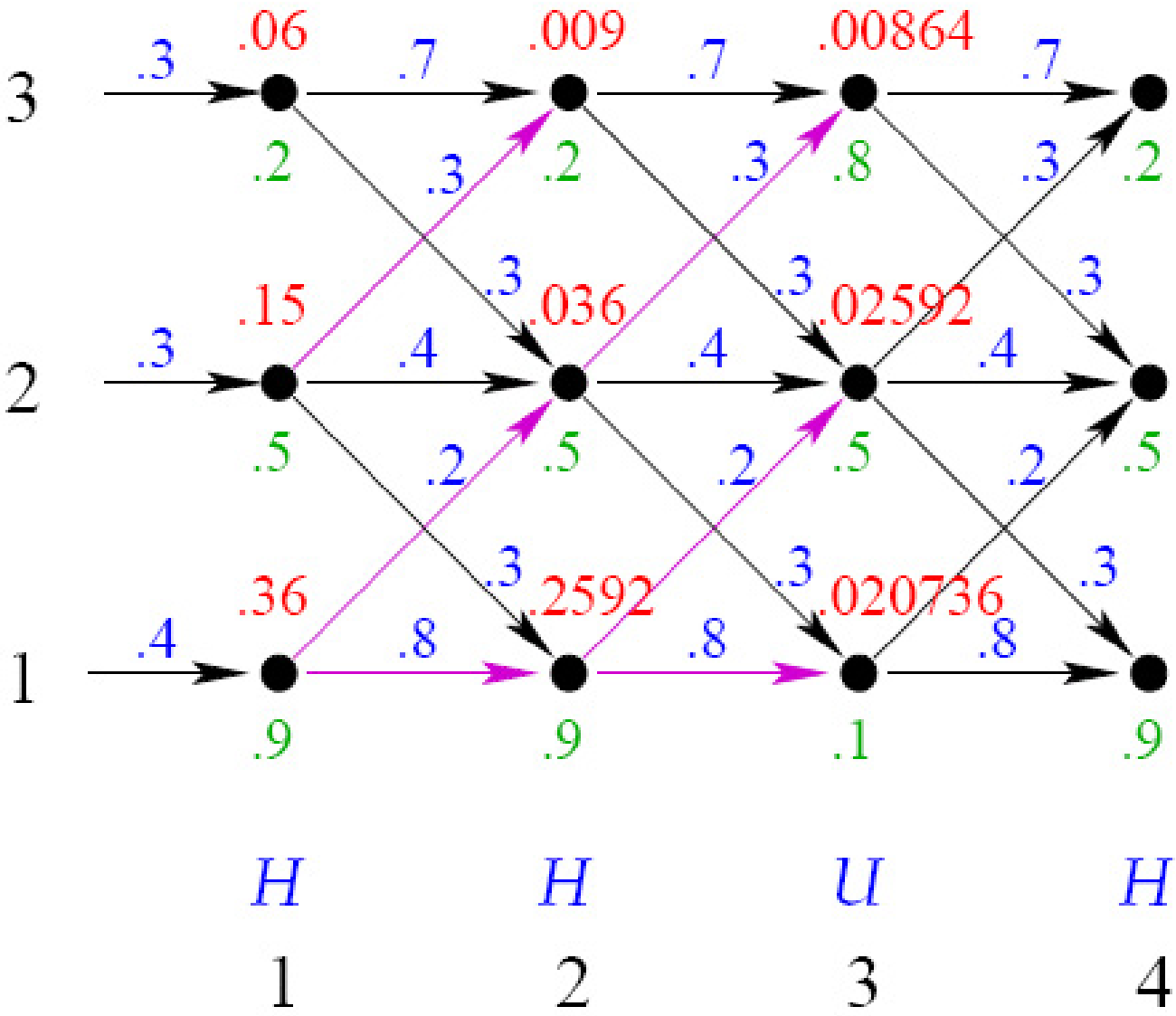
The Viterbi algorithm: Example



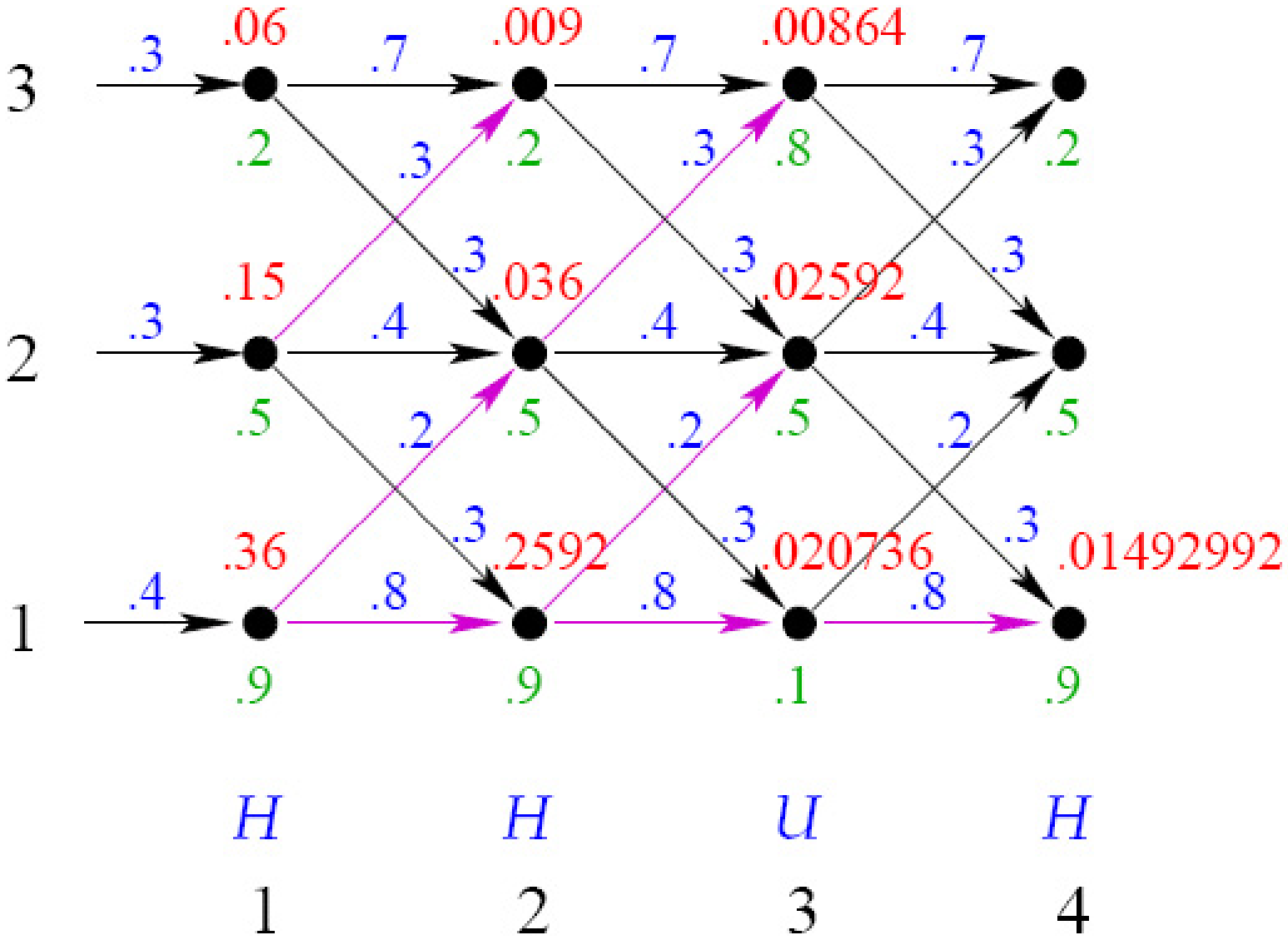
The Viterbi algorithm: Example



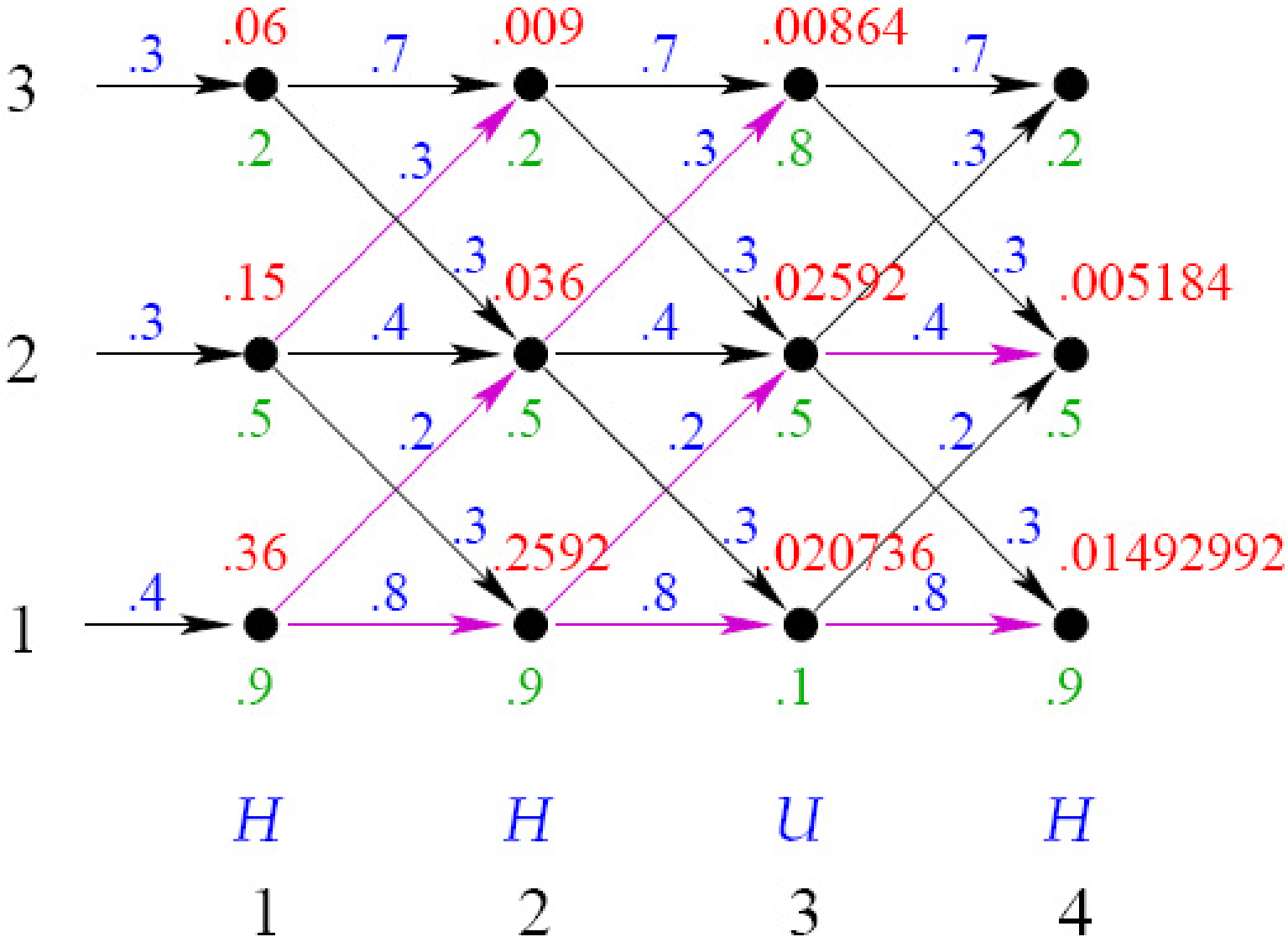
The Viterbi algorithm: Example



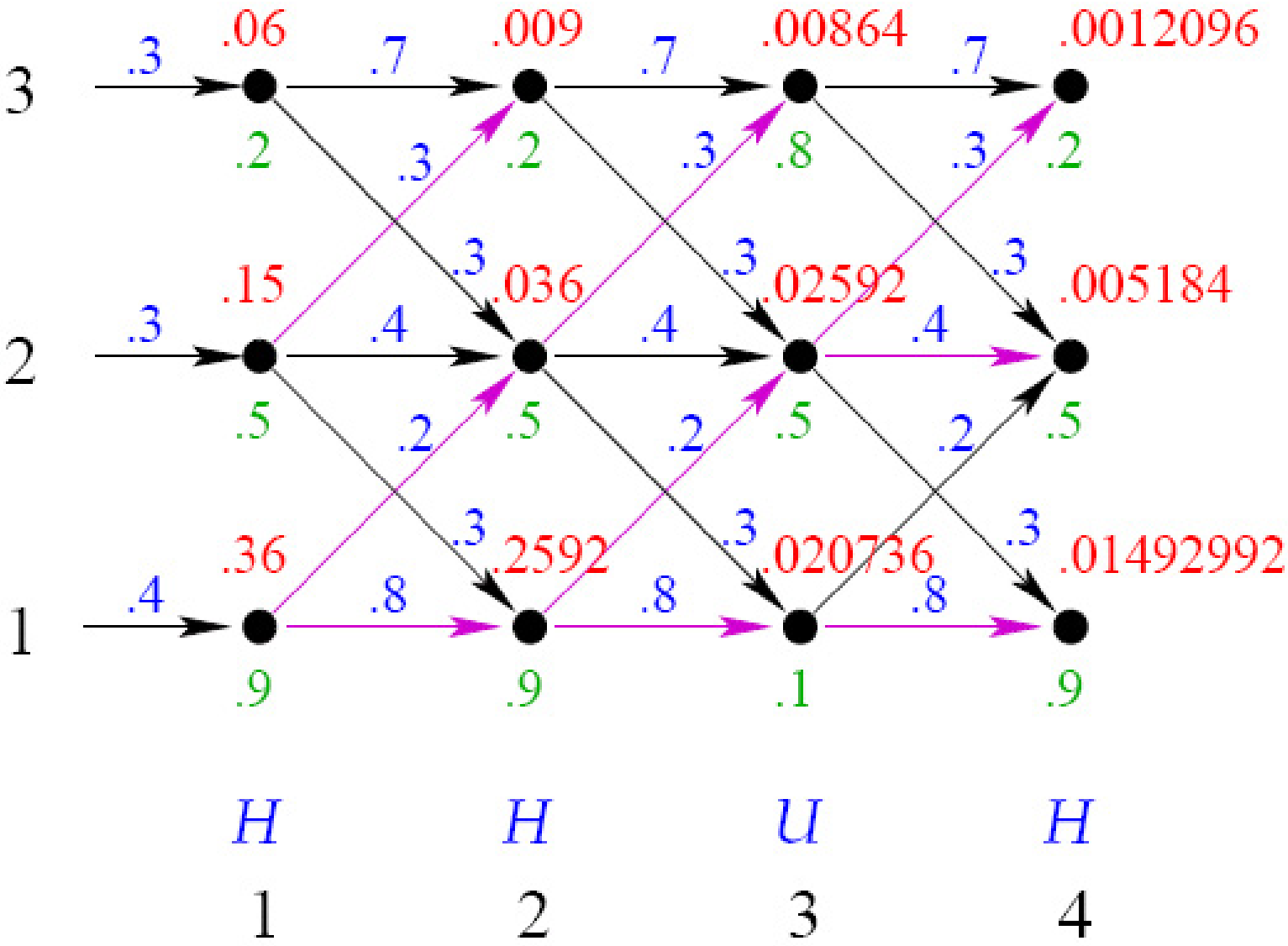
The Viterbi algorithm: Example



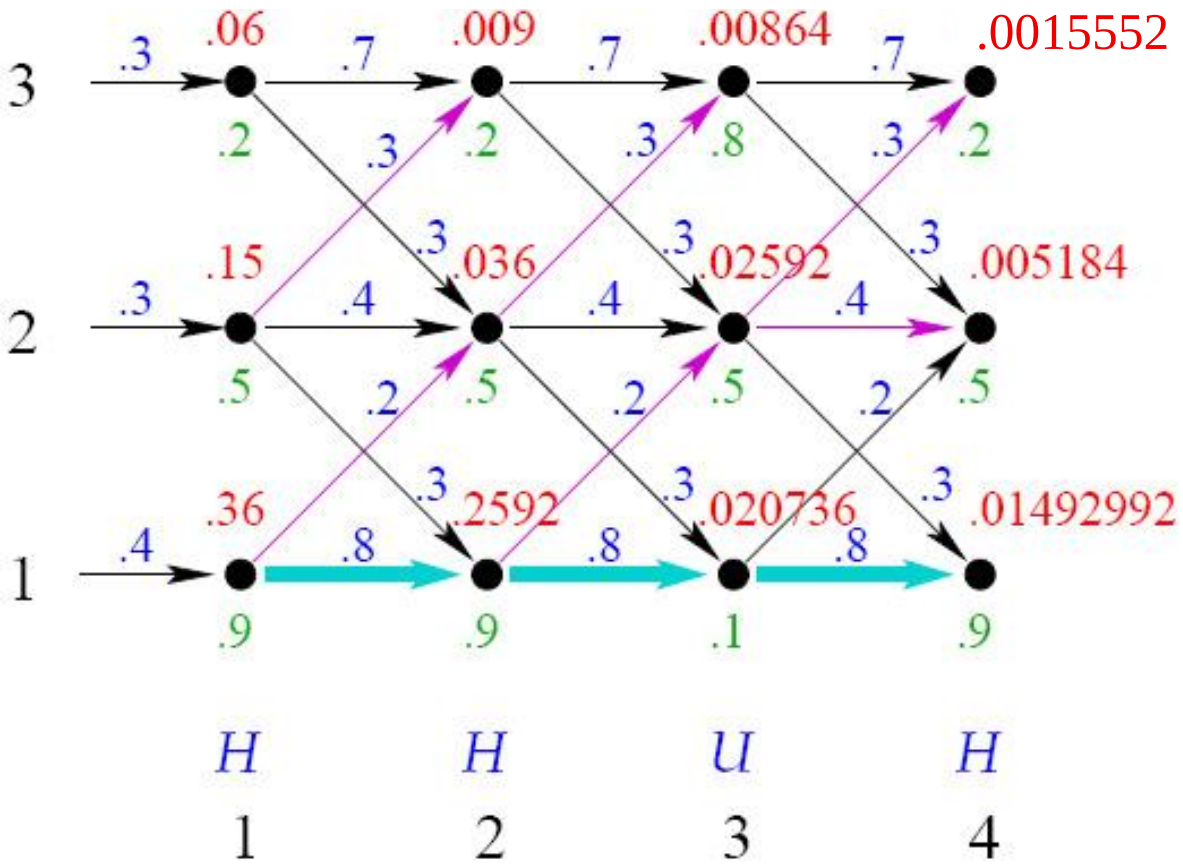
The Viterbi algorithm: Example



The Viterbi algorithm: Example



The Viterbi algorithm: Example



Solving the training problem for HMMs

- The problem: Given an observation sequence (training data), find the HMM parameters $\lambda^* = \{\pi^*, \mathbf{A}^*, \mathbf{B}^*\}$ that maximize the probability of the observations

$$\lambda^* = \underset{\lambda}{\operatorname{argmax}}\, p(\mathbf{O}|\lambda)$$

Solving the training problem for HMMs

- The problem: Given an observation sequence (training data), find the HMM parameters $\lambda^* = \{\pi^*, \mathbf{A}^*, \mathbf{B}^*\}$ that maximize the probability of the observations

$$\lambda^* = \underset{\lambda}{\operatorname{argmax}}\, p(\mathbf{O}|\lambda)$$

- If someone *gave us* the state sequence, it would be easy to find the maximum-likelihood estimates for $\mathbf{A}$ and $\mathbf{B}$:

states	1	1	1	1	2	3	3	2	3	2	2	1
observations	H	H	U	H	H	U	U	U	H	H	H	H

Solving the training problem for HMMs

- The problem: Given an observation sequence (training data), find the HMM parameters $\lambda^* = \{\pi^*, \mathbf{A}^*, \mathbf{B}^*\}$ that maximize the probability of the observations

$$\lambda^* = \underset{\lambda}{\operatorname{argmax}}\, p(\mathbf{O}|\lambda)$$

- If someone *gave us* the state sequence, it would be easy to find the maximum-likelihood estimates for $\mathbf{A}$ and $\mathbf{B}$:

states	1	1	1	1	2	3	3	2	3	2	2	1
observations	H	H	U	H	H	U	U	U	H	H	H	H

$$a_{ij} = \frac{\text{count}(i \to j)}{\text{count}(\text{transitions from } i)}$$

Solving the training problem for HMMs

- The problem: Given an observation sequence (training data), find the HMM parameters $\lambda^* = \{\pi^*, \mathbf{A}^*, \mathbf{B}^*\}$ that maximize the probability of the observations

$$\lambda^* = \underset{\lambda}{\operatorname{argmax}}\, p(\mathbf{O}|\lambda)$$

- If someone *gave us* the state sequence, it would be easy to find the maximum-likelihood estimates for $\mathbf{A}$ and $\mathbf{B}$:

states	1	1	1	1	2	3	3	2	3	2	2	1
observations	H	H	U	H	H	U	U	U	H	H	H	H

$$a_{ij} = \frac{\text{count}(i \rightarrow j)}{\text{count}(\text{transitions from } i)}$$

$$b_i(k) = \frac{\text{count}(v_k \text{ in state } i)}{\text{count}(i)}$$

Solving the training problem for HMMs (2)

- We are *not* given state alignments. Instead we will "generate" them ourselves (as we did with Gaussian component labels in GMMs)

Solving the training problem for HMMs (2)

- We are *not* given state alignments. Instead we will "generate" them ourselves (as we did with Gaussian component labels in GMMs)

- Baum-Welch Algorithm: EM for HMMs
  - Make initial guess of parameter values, then alternate between two steps:
  - E step: Compute the posterior probabilities of all states at all times, given the observations
  - M step: Compute the maximum-likelihood parameters given the *expected* numbers of occurrences of states and transitions

The Baum-Welch algorithm

- Define $\xi_t(i,j)$ as the probability of being in state i at time t and state j at time $t+1$, given the observation sequence

$$\begin{aligned}\xi_t(i,j) &= P(q_t = i, q_{t+1} = j | \mathbf{O}, \lambda) \\ &= \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{p(\mathbf{O}|\lambda)}\end{aligned}$$

The Baum-Welch algorithm

- Define $\xi_t(i,j)$ as the probability of being in state i at time t and state j at time $t+1$, given the observation sequence

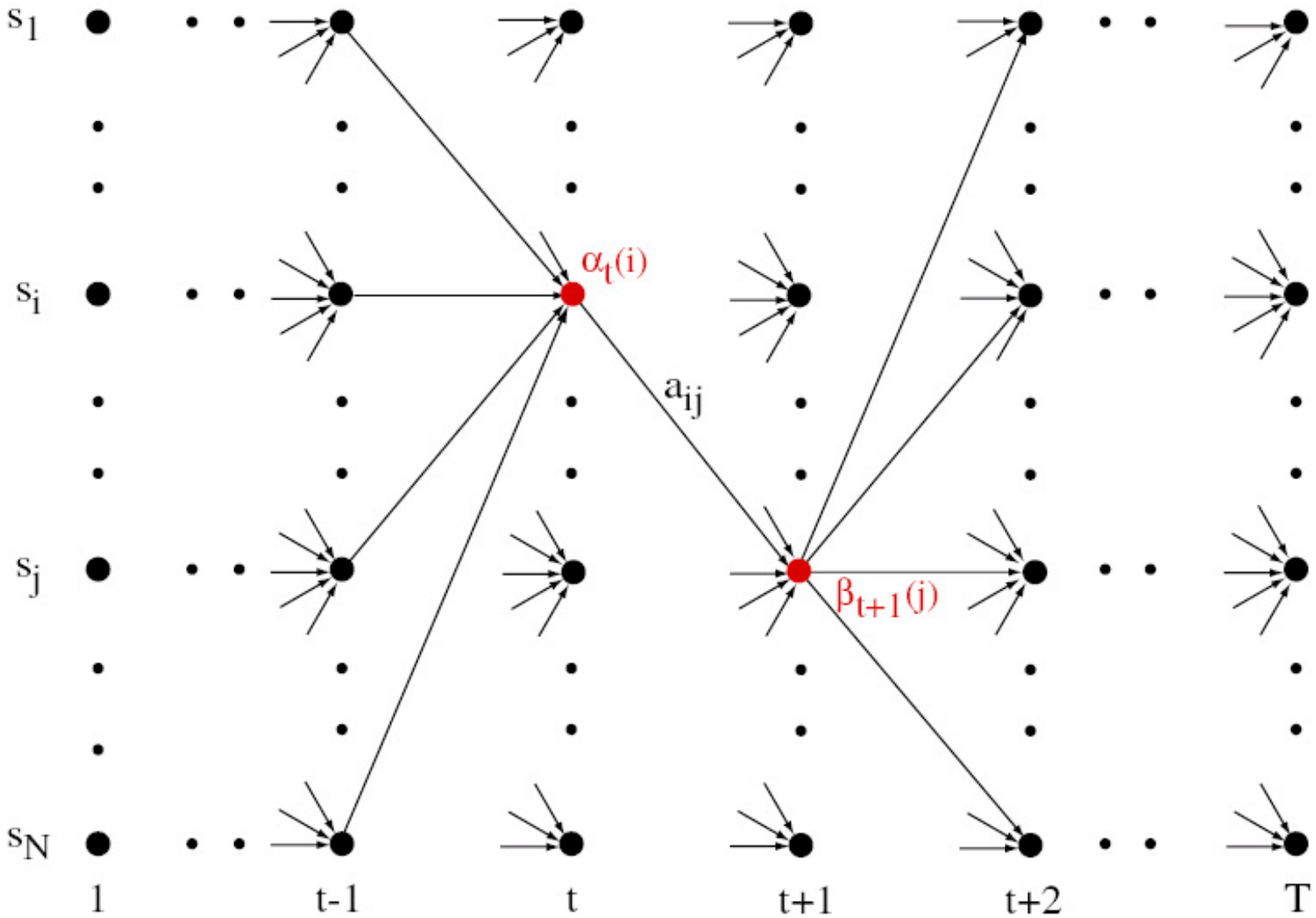
$$\begin{aligned}\xi_t(i,j) &= P(q_t = i, q_{t+1} = j | \mathbf{O}, \lambda) \\ &= \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{p(\mathbf{O}|\lambda)}\end{aligned}$$

- Then note that the state posterior probabilities are $\gamma_t(i) = P(q_t = i|\mathbf{O}, \lambda) = \sum_{j=1}^{N} \xi_t(i,j)$
- Now we can compute the *expected counts* we need, e.g.:

$$\sum_{t=1}^{T-1} \gamma_t(i) = \text{expected count of transitions from state } i$$

$$\sum_{t=1}^{T-1} \xi_t(i,j) = \text{expected count of transitions from } i \text{ to } j$$

The Baum-Welch algorithm



The Baum-Welch re-estimation formulas

$$\hat{a}_{ij} = \frac{\text{expected-count}(i \rightarrow j)}{\text{expected-count}(\text{transitions from } i)} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

$$\hat{b}_i(k) = \frac{\text{expected-count}(v_k \text{ in state } i)}{\text{expected-count}(i)} = \frac{\sum_{t=1, o_t=v_k}^{T} \gamma_t(i)}{\sum_{t=1}^{T} \gamma_t(i)}$$

$$\hat{\pi}_i = \text{expected-count}(q_1 = i) = \gamma_1(i)$$

Properties of Baum-Welch re-estimation

- If λ is the current model, and $\hat{\lambda}$ is the model after one iteration of re-estimation, then it can be shown that $p(\mathbf{O}|\hat{\lambda}) \geq p(\mathbf{O}|\lambda)$
- I.e., the likelihood is non-decreasing at each iteration
- Therefore, we can iterate the re-estimation formulas until some convergence threshold is reached (e.g. likelihood increase less than some threshold τ)
- Guaranteed to find a *local* maximum of the likelihood, but not a global one

Parameter estimation with EM

- Our maximum-likelihood parameter estimation algorithms for GMMs and HMMs are examples of the more general *expectation-maximization* algorithm:
  - Initialize parameters with a guess, then iterate until convergence:
  - Given current parameter estimates, compute posteriors of latent variables
  - Using posteriors as "counts", find maximum-likelihood estimates of parameters

Independence properties in HMMs

An HMM is a model of the joint distribution of $\mathbf{q}$ and $\mathbf{O}$:

$$\begin{aligned} p(\mathbf{O}, \mathbf{q}) = p(\mathbf{O}_{1:T}, \mathbf{q}_{1:T}) &= p(q_1)p(\mathbf{o}_1|q_1)\prod_{t=2}^{T} p(q_t|q_{t-1})p(\mathbf{o}_t|q_t) \\ &= \pi_{q_1} b_{q_1}(\mathbf{o}_q)\prod_{t=2}^{T} a_{q_{t-1}q_t} b_{q_t}(\mathbf{o}_t) \end{aligned}$$

What independence properties does this imply?

Independence properties in HMMs

An HMM is a model of the joint distribution of $\mathbf{q}$ and $\mathbf{O}$:

$$\begin{aligned} p(\mathbf{O}, \mathbf{q}) = p(\mathbf{O}_{1:T}, \mathbf{q}_{1:T}) &= p(q_1)p(\mathbf{o}_1|q_1)\prod_{t=2}^{T} p(q_t|q_{t-1})p(\mathbf{o}_t|q_t) \\ &= \pi_{q_1} b_{q_1}(\mathbf{o}_q)\prod_{t=2}^{T} a_{q_{t-1}q_t} b_{q_t}(\mathbf{o}_t) \end{aligned}$$

What independence properties does this imply?

- Notation: $X \perp \{Y, Z\}|W \Rightarrow X$ is independent of Y and Z given W

Independence properties in HMMs

An HMM is a model of the joint distribution of $\mathbf{q}$ and $\mathbf{O}$:

$$\begin{aligned} p(\mathbf{O}, \mathbf{q}) = p(\mathbf{O}_{1:T}, \mathbf{q}_{1:T}) &= p(q_1)p(\mathbf{o}_1|q_1)\prod_{t=2}^{T} p(q_t|q_{t-1})p(\mathbf{o}_t|q_t) \\ &= \pi_{q_1} b_{q_1}(\mathbf{o}_q)\prod_{t=2}^{T} a_{q_{t-1}q_t} b_{q_t}(\mathbf{o}_t) \end{aligned}$$

What independence properties does this imply?

- Notation: $X \perp \{Y, Z\}|W \Rightarrow X$ is independent of Y and Z given W
- $\mathbf{o}_t \perp \{\mathbf{q}_{\neg t}, \mathbf{O}_{\neg t}\}|q_t$, where $\mathbf{q}_{\neg t} = \{q_1, \ldots, q_{t-1}, q_{t+1}, \ldots, q_T\}$, i.e. given the current state, the current observation depends on nothing else

Independence properties in HMMs

An HMM is a model of the joint distribution of $\mathbf{q}$ and $\mathbf{O}$:

$$\begin{aligned} p(\mathbf{O}, \mathbf{q}) = p(\mathbf{O}_{1:T}, \mathbf{q}_{1:T}) &= p(q_1)p(\mathbf{o}_1|q_1)\prod_{t=2}^{T} p(q_t|q_{t-1})p(\mathbf{o}_t|q_t) \\ &= \pi_{q_1} b_{q_1}(\mathbf{o}_q)\prod_{t=2}^{T} a_{q_{t-1}q_t} b_{q_t}(\mathbf{o}_t) \end{aligned}$$

What independence properties does this imply?

- Notation: $X \perp \{Y, Z\}|W \Rightarrow X$ is independent of Y and Z given W
- $\mathbf{o}_t \perp \{\mathbf{q}_{\neg t}, \mathbf{O}_{\neg t}\}|q_t$, where $\mathbf{q}_{\neg t} = \{q_1, \ldots, q_{t-1}, q_{t+1}, \ldots, q_T\}$, i.e. given the current state, the current observation depends on nothing else
- $\{\mathbf{q}_{1:t-1}, \mathbf{O}_{1:t-1}\} \perp \{\mathbf{q}_{t+1:T}, \mathbf{O}_{t+1:T}\}|q_t$, i.e. the future is independent of the past given the current state

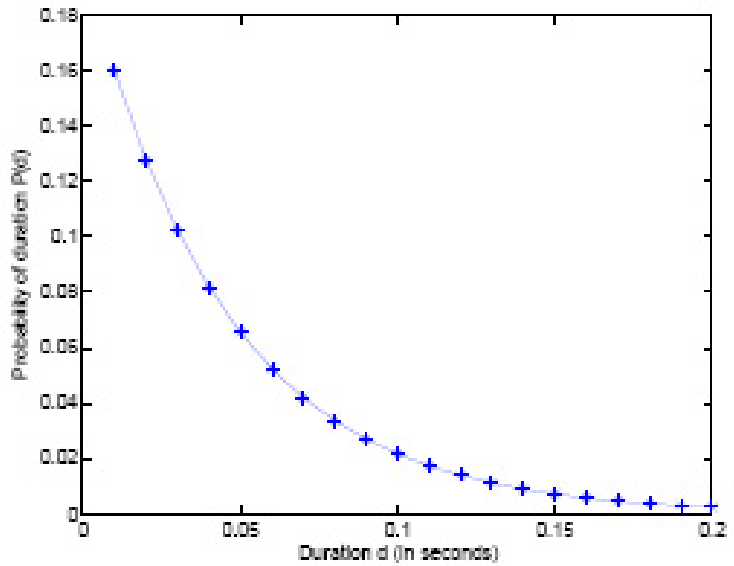
HMM extensions: State duration modeling

- The probability of staying in state i for d time steps (frames) is $a_{ii}^{d-1}(1 - a_{ii})$, a geometric distribution
- In contrast, durations of "states" we'd like to model often look nothing like that

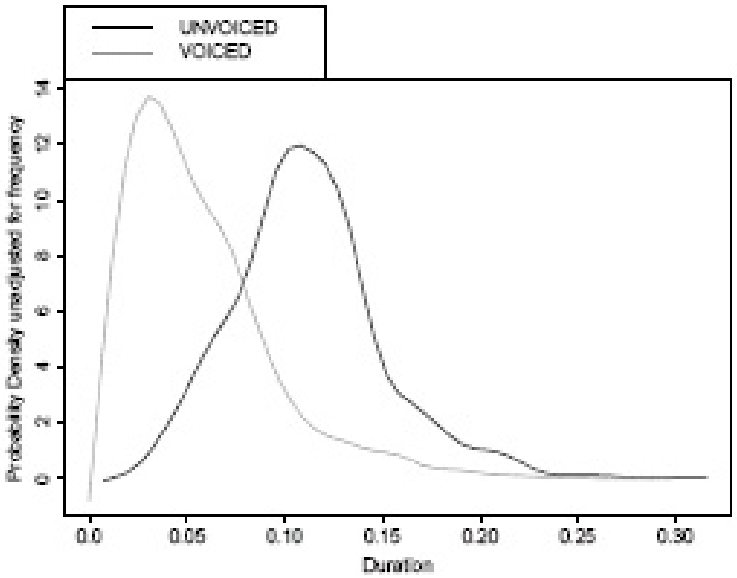
HMM extensions: State duration modeling

- The probability of staying in state i for d time steps (frames) is $a_{ii}^{d-1}(1 - a_{ii})$, a geometric distribution
- In contrast, durations of "states" we'd like to model often look nothing like that
- Example from HMMs learned on speech data vs. speech sound durations:

HMM state duration probabilities, $a_{ii} = .8$



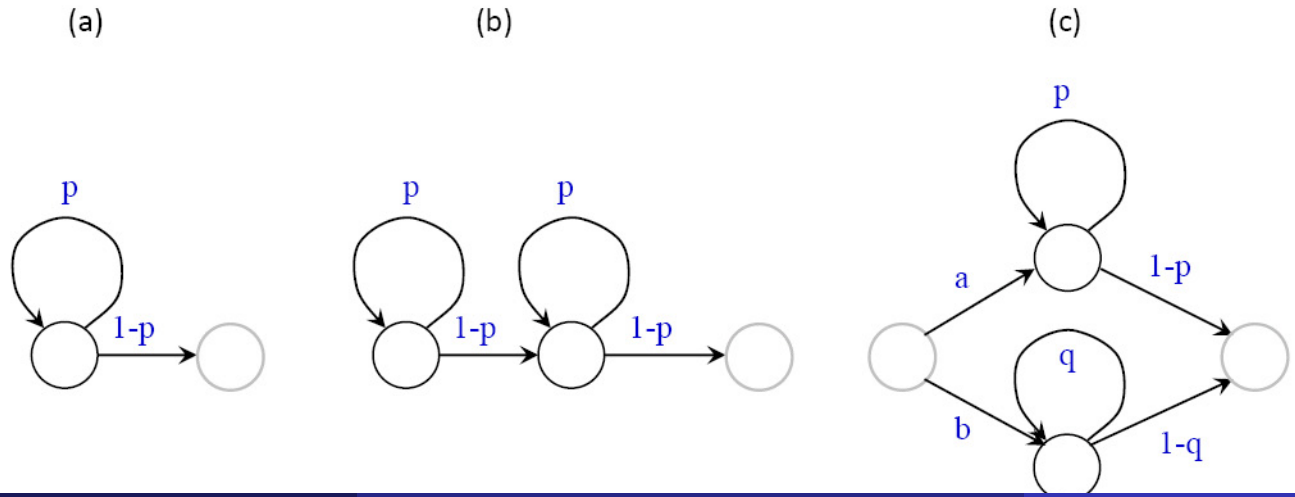
Actual fricative duration probabilities



State duration modeling

Possible solutions:

- Replace transition probabilities with explicit duration distributions [Ostendorf '96]
  - Most general approach
  - Breaks dynamic programming algorithms
- Replace single state with multiple *tied states* with identical observation distribution, and some desired transition structure
  - Simplest option: Set a minimum duration
  - Can simulate other duration distributions with alternative transition structures



HMMs with continuous observation distributions

- In a *continuous-density* HMM (CD-HMM), the discrete observation probabilities, $b_i(k)$, are replaced by continuous densities $b_i(\mathbf{o})$

HMMs with continuous observation distributions

- In a *continuous-density* HMM (CD-HMM), the discrete observation probabilities, $b_i(k)$, are replaced by continuous densities $b_i(\mathbf{o})$
- E.g., the observation distribution can be a Gaussian or mixture of Gaussians: $b_i(\mathbf{o}) = \sum_{k=1}^{K} c_{ik} \mathcal{N}(\mathbf{o}|\mu_{ik}, \Sigma_{ik}), \quad 1 \leq i \leq N$
- $c_{ik}, \mu_{ik}, \Sigma_{ik}$ are the parameters of Gaussian component k in state i

HMMs with continuous observation distributions

- In a *continuous-density* HMM (CD-HMM), the discrete observation probabilities, $b_i(k)$, are replaced by continuous densities $b_i(\mathbf{o})$
- E.g., the observation distribution can be a Gaussian or mixture of Gaussians: $b_i(\mathbf{o}) = \sum_{k=1}^{K} c_{ik} \mathcal{N}(\mathbf{o}|\mu_{ik}, \Sigma_{ik}), \quad 1 \leq i \leq N$
- $c_{ik}, \mu_{ik}, \Sigma_{ik}$ are the parameters of Gaussian component k in state i
- Alternatively, observation distribution can be modelled by a neural network (referred to as "hybrid HMM/NN" in the speech literature)
- Doesn't change anything in the algorithms; just replace $b_i(k)$ by the value of the corresponding density

Semi-supervised learning
(Figure credits: Zhu ICML 2007 tutorial unless otherwise indicated)

Combining supervised and unsupervised training

- We often have access to a small amount of labeled data plus a large amount of unlabeled data
- Why not just get more labeled data?
  - Some labeled data is easy to get more of, e.g. tagged images
  - Some is expensive/time consuming/difficult/unethical to get:
    - Medical measurements (e.g. scans, neural recordings)
    - Image segmentation: requires careful manual effort
    - Phonetic transcription of speech: roughly 400x real time, requires experts

Combining supervised and unsupervised training

- We often have access to a small amount of labeled data plus a large amount of unlabeled data
- Why not just get more labeled data?
  - Some labeled data is easy to get more of, e.g. tagged images
  - Some is expensive/time consuming/difficult/unethical to get:
    - Medical measurements (e.g. scans, neural recordings)
    - Image segmentation: requires careful manual effort
    - Phonetic transcription of speech: roughly 400x real time, requires experts
- Today: Survey of some semi-supervised learning techniques
- We will mainly focus on classification tasks

Notation

(Borrowed from Jerry Zhu)

- Input x, label y
- Function to be learned $f : \mathcal{X} \longrightarrow \mathcal{Y}$
- Labeled data $(X_l, Y_l) = \{(x_{1:l}, y_{1:l})\}$
- Unlabeled data $X_u = \{x_{l+1:n}\}$
- Typically, $l \ll n$
- Test data $X_{test} = \{x_{n+1:}\}$

Terminology

Several settings combining labeled and unlabeled data

Terminology

Several settings combining labeled and unlabeled data

- **Transductive learning**: Goal is to label the unlabeled training data (only)

Terminology

Several settings combining labeled and unlabeled data

- **Transductive learning**: Goal is to label the unlabeled training data (only)
- **Inductive learning (semi-supervised learning)**: Goal is to learn a function that can be applied to new test data (i.e., has an out-of-sample extension)

Terminology

Several settings combining labeled and unlabeled data

- **Transductive learning**: Goal is to label the unlabeled training data (only)
- **Inductive learning (semi-supervised learning)**: Goal is to learn a function that can be applied to new test data (i.e., has an out-of-sample extension)
- **Weak labels/distant supervision**: We have labels, but they are not exactly for the target task

Terminology

Several settings combining labeled and unlabeled data

- **Transductive learning**: Goal is to label the unlabeled training data (only)
- **Inductive learning (semi-supervised learning)**: Goal is to learn a function that can be applied to new test data (i.e., has an out-of-sample extension)
- **Weak labels/distant supervision**: We have labels, but they are not exactly for the target task
  - "Same/different" pairs, instead of class labels

Terminology

Several settings combining labeled and unlabeled data

- **Transductive learning**: Goal is to label the unlabeled training data (only)
- **Inductive learning (semi-supervised learning)**: Goal is to learn a function that can be applied to new test data (i.e., has an out-of-sample extension)
- **Weak labels/distant supervision**: We have labels, but they are not exactly for the target task
  - "Same/different" pairs, instead of class labels
  - Subset of latent variables labeled but not all (e.g. object category in an image, but not boundaries)

Terminology

Several settings combining labeled and unlabeled data

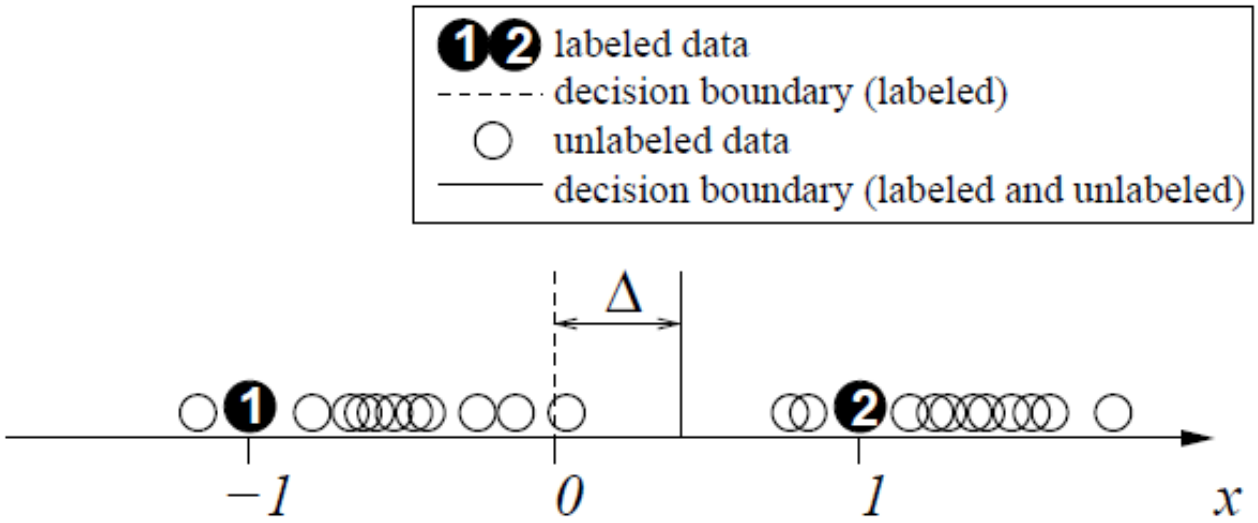
- **Transductive learning**: Goal is to label the unlabeled training data (only)
- **Inductive learning (semi-supervised learning)**: Goal is to learn a function that can be applied to new test data (i.e., has an out-of-sample extension)
- **Weak labels/distant supervision**: We have labels, but they are not exactly for the target task
  - "Same/different" pairs, instead of class labels
  - Subset of latent variables labeled but not all (e.g. object category in an image, but not boundaries)
  - "Downstream" labels (e.g. click-through counts instead of search result correctness)

Motivation

If we have labeled data, how can unlabeled data help?

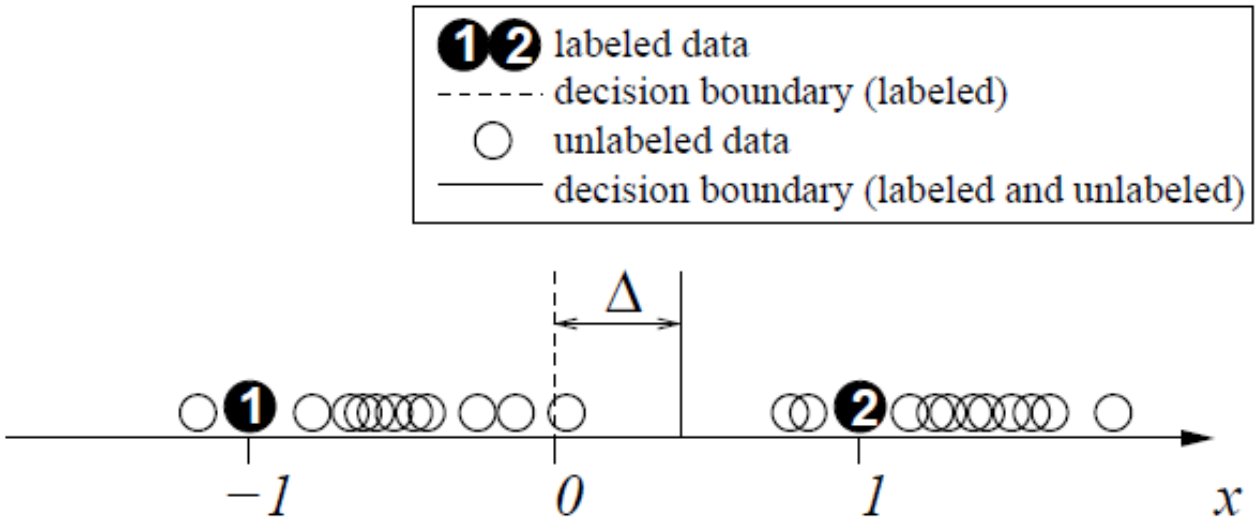
Motivation

If we have labeled data, how can unlabeled data help?



Motivation

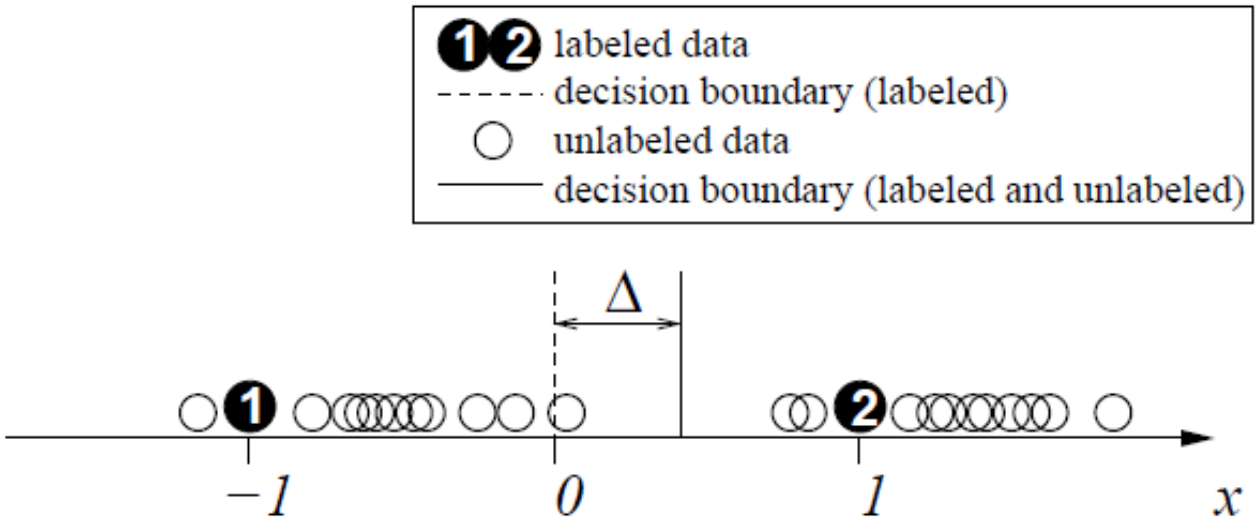
If we have labeled data, how can unlabeled data help?



Unlabeled data tells us something about the distribution of the data, and therefore where "reasonable" class boundaries may be

Motivation

If we have labeled data, how can unlabeled data help?

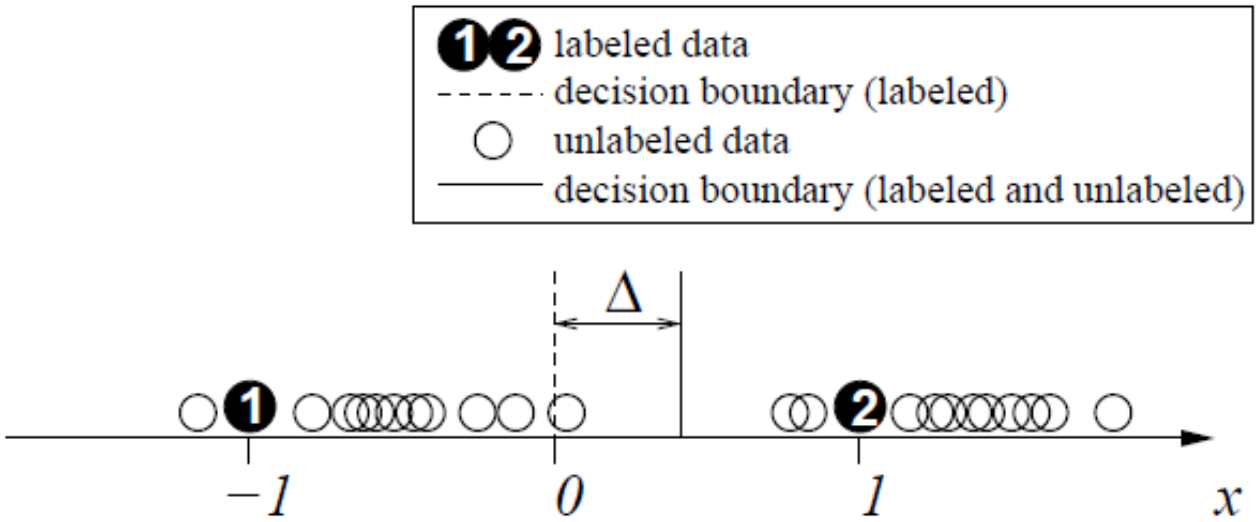


Unlabeled data tells us something about the distribution of the data, and therefore where "reasonable" class boundaries may be

(**Note**: The true boundaries **could** be "unreasonable"... but why would the data generator do that to us?)

Motivation

If we have labeled data, how can unlabeled data help?



Unlabeled data tells us something about the distribution of the data, and therefore where "reasonable" class boundaries may be

(**Note**: The true boundaries **could** be "unreasonable"... but why would the data generator do that to us?)
Evidence from human learning: Babies seem to be very successful at semi-supervised learning (e.g., [LaTourette & Waxman 2019])

Techniques for combining labeled and unlabeled data

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data – like on the HW)

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data – like on the HW)
- Self-training (first model labeled data, then unlabeled data)

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data – like on the HW)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data – like on the HW)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)
- Graph-based algorithms

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data – like on the HW)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)
- Graph-based algorithms
- Multi-view algorithms

Self-training

Self-training algorithm:

1. Train f from (X_l, Y_l)
2. Predict on $x \in X_u$
3. Add $(x, f(x))$ to labeled data
4. Repeat

Self-training

Self-training algorithm:

1. Train f from (X_l, Y_l)
2. Predict on $x \in X_u$
3. Add $(x, f(x))$ to labeled data
4. Repeat

- This is a type of "wrapper" technique: can be used with any type of learning algorithm for $f(\cdot)$
- Typically, only $(x, f(x))$ pairs with high confidence are used

Self-training example

1. Train a classifier on small number of labeled images of two classes, **eclipse (astronomy)** and **eclipse (car)**

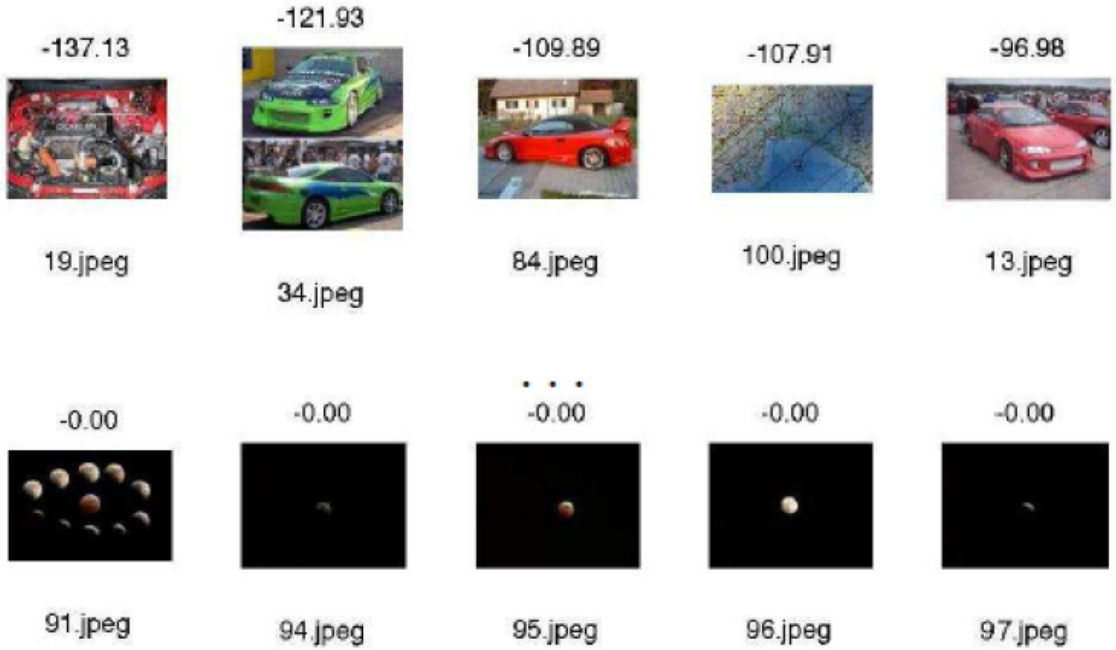


Self-training example

1. Train a classifier on small number of labeled images of two classes, **eclipse (astronomy)** and **eclipse (car)**

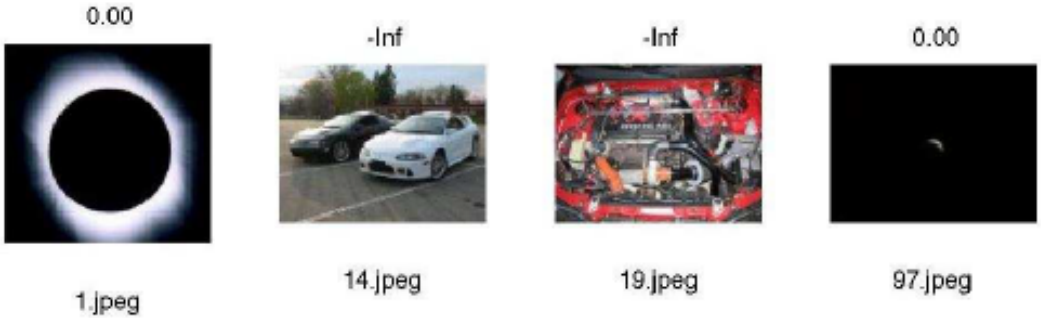


2. Classify unlabeled data, sort by confidence $\log p(y = \text{astronomy}|x)$

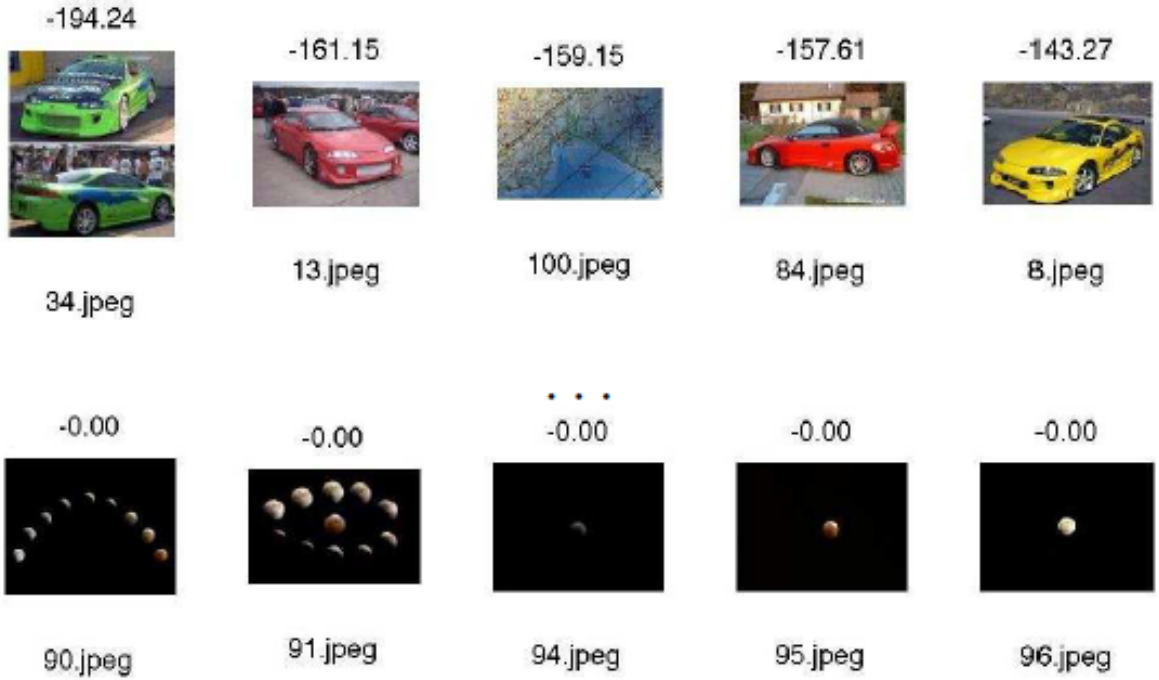


Self-training example

3. Add the most confident images and predicted labels to labeled data



4. Re-train the classifier and repeat



Self-training

Advantages

- Very simple
- A "wrapper" method: Can be applied with any existing classifier
- Has had some real-world successes (e.g. in speech and NLP)

Self-training

Advantages

- Very simple
- A "wrapper" method: Can be applied with any existing classifier
- Has had some real-world successes (e.g. in speech and NLP)

Disadvantages

- Very risky: Can easily diverge if some early mistakes are made
- Little theoretical understanding